Hotel Loyalty Points Mall

Software Design Report

CS6451: Advanced Software Design • Autumn 2026–2027
University of Limerick

Team-Based Project

Lecturer: J. J. Collins

Submission: Week 8

Team Members

Fude Ding	26046091
Kewei Wu	26141825



Contents

1 Business Scenario **1**

2 Software Lifecycle **2**

3 Project Plan **3**

3.1 Roles and Responsibilities . . . 3

3.2 Team Experience . . . 3

4 Requirements **4**

4.1 Functional . . . 4

4.1.1 Customer Service . . . 4

4.1.2 Manager . . . 4

4.1.3 Use Case Diagrams . . . 4

4.1.4 Use Case Descriptions . . . 6

4.2 non-functional . . . 7

4.2.1 User-Friendliness . . . 7

4.2.2 Concurrency . . . 7

4.2.3 Response time . . . 7

4.2.4 Implementation Strategy . . . 7

4.2.5 Quality Tactics . . . 7

4.3 GUI Prototypes . . . 8

5 System Architecture **9**

5.1 Overview . . . 9

5.1.1 Architectural Pattern 1: Model-View-Controller (MVC) . . . 9

5.1.2 Architectural Pattern 2: Publish-Subscribe with Redis . . . 9

5.2 Common Services and Rules . . . 10

5.3 Technology Pipeline . . . 10

5.4 High-level Package Diagram . . . 11

6 Conceptual Design **12**

6.1 List of Candidate Objects . . . 12

6.2 Sketch of Class Diagram . . . 12

6.2.1 Analysis Class Diagram – Core Model . . . 12

6.2.2 Analysis Diagram – Create Booking Flow . . . 13

6.2.3 Analysis Diagram – Check-in Flow . . . 13

6.3 Sequence Diagrams . . . 14

6.3.1 Searching Room . . . 14
6.3.2 Booking Room . . . 14
6.3.3 Payment . . . 15
6.4 State Chart Diagram . . . 15
6.5 Entity Relationship Diagram with Cardinality . . . 16

7 Transparency And Traceability **17**
7.1 Classes Per Package . . . 17
7.2 LOC Per Author . . . 17

8 Implementation **19**
8.1 Architectural Patterns . . . 19
8.1.1 MVC Architectural Pattern . . . 19
8.1.2 Layered Architectural Pattern . . . 20
8.1.3 Interceptor Architectural Pattern . . . 20
8.1.4 Built-in Interceptor . . . 24
8.1.5 Dependency Injection Architectural Pattern . . . 26
8.2 Design Pattern . . . 26
8.2.1 Observer Design Pattern . . . 26
8.2.2 Factory Design Pattern . . . 29
8.2.3 Decorator Design Pattern . . . 30
8.2.4 State Design Pattern . . . 32
8.2.5 Strategy Design Pattern . . . 33
8.3 Unit Test . . . 35
8.4 Version Control . . . 36

9 Added Value **38**
9.1 Continuous Integration and Continuous Delivery . . . 38
9.1.1 Docker . . . 38
9.1.2 GitHub Actions . . . 38
9.2 Pattern-based payment module . . . 38
9.2.1 Description . . . 38
9.2.2 Implementation . . . 39
9.3 Security and OWASP-inspired hardening . . . 42
9.3.1 Description . . . 42
9.3.2 Implementation . . . 42
9.4 Database support and ORM . . . 45
9.4.1 Description . . . 45

9.4.2 Implementation . . . 46
9.5 Monolith Architecture Pattern . . . 48
9.5.1 Description . . . 48
9.5.2 Implementation . . . 48
9.6 Google Login . . . 48
9.6.1 Description . . . 48
9.6.2 Implementation . . . 48

10 Recovered Architecture And Design Blueprints **49**
10.1 Design-time Package Diagram . . . 49
10.2 Design-time Class Diagram . . . 50
10.2.1 Core entities . . . 50
10.2.2 Core services . . . 50
10.2.3 Payment patterns . . . 51
10.2.4 Pricing engine and order updates . . . 52
10.3 Sequence Diagram . . . 53
10.3.1 Searching Room . . . 53
10.3.2 Booking Room . . . 53
10.3.3 Payment Process . . . 54
10.4 Deployment Diagram . . . 54
10.5 Component Diagram . . . 55

11 Evaluation **56**

12 Reflection **57**
12.1 Ismail Bahlaoui . . . 57
12.2 Dechao Zhan . . . 57
12.3 Tan Le . . . 57
12.4 Wenkai Zhu . . . 58

13 Use of Generative AI **59**

1 Business Scenario

The Automated Hotel Reservation System is a lightweight proof-of-concept for ABCD Hotels. It modernises booking and stay management with full self-service via web and kiosk. The design uses clear business rules and a modular structure that can grow over time. External payment, notification, and kiosk services are connected as mock adapters and keeping a clear path to real integrations.

Two roles are in scope, Guest and Manager. Guests sign in with username and password or google account. Access is role-based, and logs hide sensitive data such as card numbers email. Guests search with filters like dates, room type, and price, receive dynamic price quotes, and create bookings with online or offline payment. They can view, modify, and cancel bookings. A membership page shows the guest's tier status such as Bronze, Silver, or Gold and current applied promotions. Self check-in uses booking code plus ID verification, the system allocates a room, processes the due amount, and issues a QR code key. Self check-out calculates the final bill, processes payment, and deactivates the QR key. High tier members may check-in early when a room is ready or check-out late when they are not ready. Managers view and update room status such as vacant, occupied, needs cleaning, or maintenance. They handle kiosk exceptions like code mismatches or payment issues. They configure pricing strategies like rate plans, promotions, coupons, blackout dates, and membership benefits, managers create reports on occupancy, revenue, and promotion outcomes, and they examine audit logs to ensure accountability.

Core logic covers multi-hotel availability, a dynamic pricing engine that calculates room quotes using seasonality, occupancy levels, membership tiers, corporate agreements, and valid coupons. During booking, a temporary countdown lock prevents double booking and auto releases on timeout or failure, also a controlled overbooking margin is supported. Policies include deposits, cancellation windows with fee tiers, and no-show charges.

The system keeps an encrypted log for managers only. New data can be added but existing data cannot be modified or deleted, this encrypted log tracks the states Created, Reserved/Confirmed, Checked-In, Checked-Out, Cancelled, and No-Show. The system sets clear quality targets. It should handle at least 100 users at the same time. The server must respond to availability checks, price quotes, and actions in the main page within two seconds. To achieve this, the design uses efficient date-range checks and simple caching. It also applies clustering and failover for fast recovery. The backend uses Java/JVM, so it runs on web and kiosk channels.

2 Software Lifecycle

The software lifecycle gives a clear way to deliver quality software. We use an iterative model with short cycles that include planning, requirements, architecture and design, implementation, testing, and review. Weekly reviews keep the scope focused and give fast feedback. This is suitable for a proof of concept because it allows change when new information appears while we keep attention on agreed outcomes and deliverables across web and kiosk.

The process stays flexible yet controlled. We refine requirements in small steps and trace them to design and tests so intent is clear. We present design choices with simple diagrams and short notes, and we keep the architecture easy to read as it grows. Implementation moves in vertical slices that join user interface, logic, and data. This supports early demos and informed decisions. Testing covers unit and integration checks to confirm that main functions work as expected. At the end of each cycle we update plans based on what we learned and what is our next plan.

The requirements set clear targets and guide the work. We focus on performance, concurrency, availability, security, extensibility, and portability. We choose the methods early and use them in a consistent way. We track a small set of metrics to measure progress. We keep evidence of results, design updates, and test outcomes so the record is clear and the report matches the system.

Project control focuses on clarity and small safe steps. The system is split into well defined parts that work together with low coupling. We plan work in small complete parts that show progress and support change. Interfaces between parts have clear contracts so updates do not break other areas. External systems are reached through adapters for payment, notification, and kiosk. This reduces risk and keeps options open for future providers. Version control, shared documents, and trace tables give one view of tasks, decisions, and contributions. Small weekly improvements keep the structure clean.

We use Agile instead of Waterfall because the project is a proof of concept and we must learn fast. Agile gives frequent feedback, early demos, and steady updates as we refine features. Waterfall would fix the scope too early and delay feedback from mock adapters, which increases risk.

Coding style rules keep the code base clear and easy to maintain. Methods stay under twenty lines, and classes stay under two hundred lines. Names are meaningful, so the code reads by itself. All public APIs include JavaDoc. During design and refactoring we follow SOLID principles.

To make the structure clear, we use simple naming rules. Controllers end with Controller. Service interfaces end with Service, and their implementations end with ServiceImpl. Repositories end with Repository. Configuration classes end with Config. Data transfer models use the suffix DTO or Request-Response.

We handle errors in a clear and safe way. We use specific exceptions and do not hide errors. A global handler in the web layer turns exceptions into clear messages with the correct HTTP status codes. Logs never show sensitive data, they contain only masked values and a trace id. When a failure happens, the system releases any temporary booking lock and rolls back partial work to prevent double booking. Errors from adapters are wrapped in domain exceptions and marked as retriable or not retriable to guide recovery and alerts.

Version control supports safe change. Work happens on feature branches such as “feature/userAuthentication”. Commits are small and have clear messages such as “feat: add user login endpoint”. Secrets and IDE files are not committed. Changes merge to the main line after review to keep the code stable.

These lifecycle practices support the non functional goals and match the modular architecture and adapter based integrations for ABCD Hotels. They help us deliver a secure, responsive, and reliable system.

3 Project Plan

3.1 Roles and Responsibilities

Table 1: Team Roles and Responsibilities

No.	Role	Description	Designated Member
1	Project Manager	Sets up group meetings, agrees on the project plan, and tracks progress.	Ismail Bahlaoui
2	Documentation Manager	Sources relevant supporting documentation and composes it into the report.	All Members
3	Business Analyst / Reqs. Engineer	Responsible for Section 4 – Requirements.	Wenkai Zhu
4	Architect	Defines the system architecture.	Ismail Bahlaoui
5	Systems Analyst	Creates the conceptual models.	Wenkai Zhu
6	Designer	Recovers design-time blueprints from implementation.	Dechao Zhan
7	Technical Lead	Leads the implementation effort.	Dechao Zhan
8	Programmers	Each team member develops at least one package in the architecture.	All Members
9	Tester	Develops and runs automated test cases.	Tan Le
10	DevOps	Ensures each team member is competent with development infrastructure.	Tan Le

3.2 Team Experience

Table 2: Team Experience Summary

Member	Years	Domains	Technologies
Tan Le	6	Banking; Workforce; E-commerce	Java; Spring
Dechao Zhan	7	Software Development; Android	Java; Kotlin; Flutter; Android Toolkits
Ismail Bahlaoui	2	Software Development; E-commerce	Java; Spring; Python
Wenkai Zhu	5	E-Government Consulting	Java; Spring

4 Requirements

4.1 Functional

4.1.1 Customer Service

- System should support customer login via standard username/password or a Google account.
- Customer could be able to search for available rooms. The system should provide convenient filtering capabilities (Dates, Room type, Price range).
- Customer could be able to create new room booking requests, including basic information, selecting online payment.
- Customer could be able to view details of their bookings (past & current).
- Customer could be able to cancel or modify their requests.
- Customer could be able to access the membership page to view their current membership tier info and all exclusive member benefits. Also, customer should be able to view details of all currently available promotions, including tier-based discounts, coupons, and special event discounts.
- The customer could be able to complete the check-in process by a self-service kiosk. This process requires entering and validating the digital verification code received at the time of booking. After successful verification, the system automatically allocates a room, processes any payments due at check-in, and prints a QR code as the room key.
- The customer should be able to complete the check-out process via a self-service kiosk. The system shall calculate the final bill (including remaining room charges, additional consumption, etc.), process self-service payment for check-out, and deactivate the QR code room key.

4.1.2 Manager

- Manager could be able to view the current status of all hotel rooms (vacant, occupied, needs cleaning, under maintenance, etc.) and manually update room status if necessary.
- Manager could be able to handle exceptions of the self-service check-in process (code mismatches, offline payment errors, etc.).
- Manager could be able to handle exceptions of the self-service check-out process, collecting all outstanding charges (remaining room balance, additional consumption, damage compensation, etc.).
- Manager could be able to configure and manage the hotel's pricing strategies, including managing core pricing policies (rate plans, dynamic adjustment parameters), managing short-term revenue strategies (promotions, coupons, blackout dates), and managing the definition and benefits of membership tiers.
- Manager could be able to view and generate various key business reports, including occupancy reports, revenue reports, and promotion performance reports, to support business decision-making.
- Manager could be able to access and view system audit logs.

4.1.3 Use Case Diagrams

- Customer Service

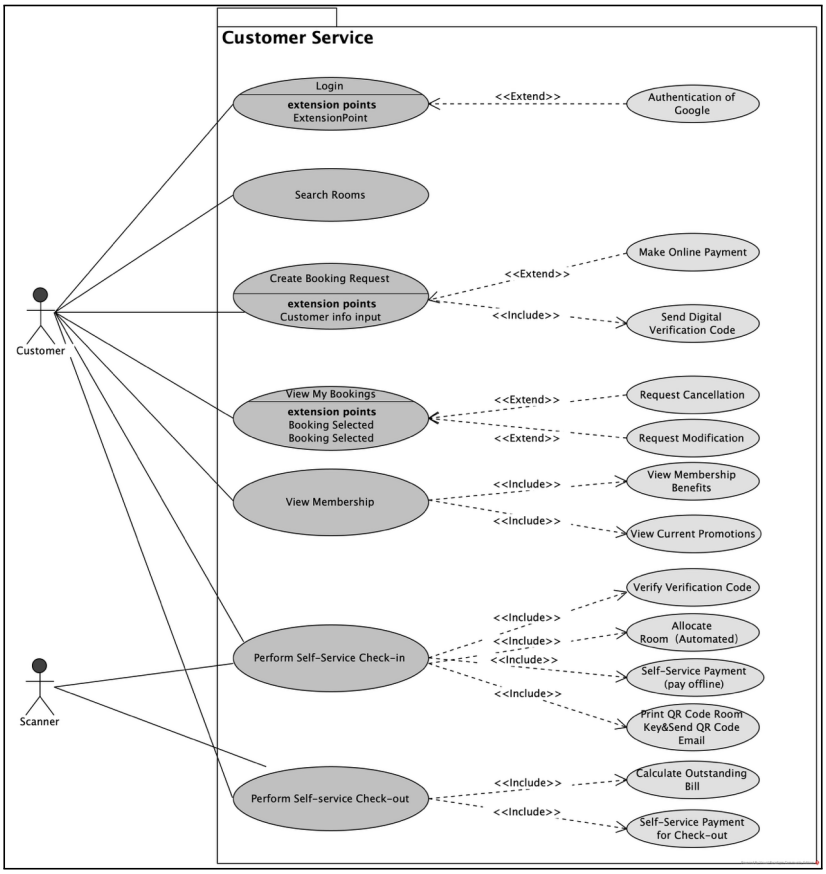


Figure 1: Customer Service Use Case

- Hotel Management

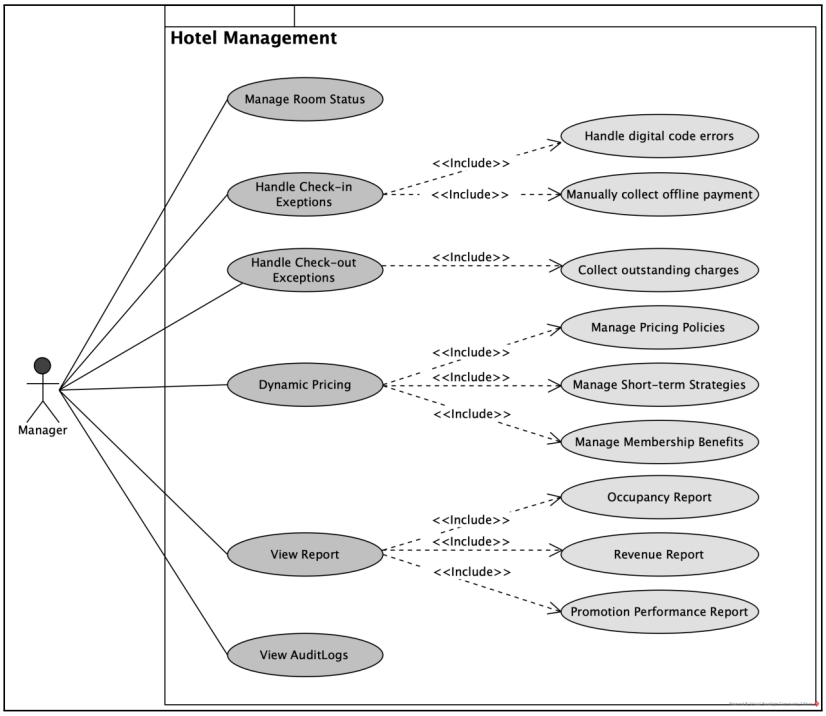


Figure 2: Hotel Management Use Case

4.1.4 Use Case Descriptions

- Create Booking Request

USE CASE	Create Booking Request	
Goals in Context	Booking a desired hotel room and to receive a confirmation and credential for the self-check-in process.	
Scope & Level	Customer Service,System	
Preconditions	Customer has already accessed the booking interface of Hotels via the website, self-service kiosk.	
Success End Conditions	System successfully created a permanent reservation record with the status "Confirmed" (pay online) or "reserved" (pay offline).	
	Customer receives a unique reservation code for self-check-in.	
	The room inventory is updated within the specified date.	
Failed End Condition	Booking is not created.	
	The room inventory remains unchanged.	
	The payment process is not completed or fails.	
Actors	Primary: Customer.	
	Secondary: Payment Gateway (mock), Email Service (mock).	
Trigger	Customer submits search criteria such as number of people, dates, to initiate the booking process.	
Description	**Step**	**Action**
	1	Customer selects search criteria including hotel location, check-in/out date, and number of people.
	2	System checks room availability and filters for room types.
	3	System invokes the dynamic pricing engine to calculate price based on seasons, membership tiers, and corporate discounts.
	4	System displays a list of available rooms and their prices.
	5	Customer selects a room and proceeds with booking.
	6	System sets a temporary countdown lock on the selected room and sets its status to "pending" until payment is completed.
	7	Customer inputs personal information required for the booking (name, phone number, etc.).
	8	Customer selects payment option (offline/online) and submits.
	9	Customer confirms information and clicks the “Complete Booking” button.
	10	System sends a payment request to the mock Payment Provider.
	11	Payment provider returns a successful payment result.
	12	System creates a booking record in the database with status “Confirmed” and generates a unique booking code.
	13	System sends a confirmation email with booking details and confirmation code.
	14	System displays the successful confirmation message on the front-end.
Extensions	2a	No available rooms: notify customer and suggest changing dates or room type.
	6a	Customer abandons booking: if not completed in time, system releases room after timeout.
	11a	Payment failure: system notifies customer and allows retry or alternate method, releases room after timeout.
Variations	3a	Use of a coupon: customer enters a valid coupon code; system validates and applies discount.
	3b	Membership discount: system applies discount automatically based on membership tier (Gold, Silver, Bronze).
Due Date	Week 6	

Table 3: Use Case: Create Booking Request

- Perform Self Check-in

USE CASE	Perform Self Check-in	
Goals in Context	Using a self-service kiosk to quickly complete the check-in process and obtain a room key.	
Scope & Level	Customer Service,System	
Preconditions	The Customer has arrived at the hotel and is at a self-service kiosk. The Customer has already got a valid booking code obtained through the booking process.	
Success End Conditions	The booking status is updated to "checked-in".	
	The customer receives a QR code room key which is valid for the duration of the booking.	
Failed End Condition	Check-in procedures fail to complete.	
	The booking status is not updated.	
Primary, Secondary, Actors	Primary: Customer. Secondary: self check-in/out machine, notification service.	
Trigger	Customer arrives at the hotel and clicks the "check-in" button on machine to start check-in.	
Description	**Step**	**Action**
	1	The Customer selects the "Check-in" button on the kiosk.
	2	Customer enters their booking code and system validates the booking code. Customer scans or enters their identification (such as passport), and system compares the customer's identity information with the information registered at the time of booking.
	3	If code is successfully validated, and the comparison of identity succeed, system will generate a time-limitated QR code as the room key.
	4	The system prints the QR code room key to the customer and also sends the QR code to the customer's email.
	5	The system updates the status of the booking record to "Checked-In".
Extensions	2a	Booking code invalid or not found: If the booking code is incorrect, the system will display an error message and advise the customer to check and retry.
	2b	Identification verify unsuccessfully: If the identity information does not match the booking, the system will refuse the check-in and advise the customer to seek manual assistance.
Variations	2a	High-tier members can check in up to two hours early if their room is ready. Otherwise, the system shows when it will be available.
Due Date	Week6	

Table 4: Use Case: Perform Self Check-in

4.2 non-functional

4.2.1 User-Friendliness

All user interfaces, including the self-service interface for customers and the management back office for in-house managers, must be designed to be simple, intuitive and easy to use. Information presentation must be clear and unambiguous to ensure that users can complete their tasks efficiently and accurately.

4.2.2 Concurrency

The system must be able to support at least 100 concurrent users performing simultaneous operations (such as searching, booking) without a significant degradation in performance.

4.2.3 Response time

For critical backend processing requests, such as "availability checks" and "price quote calculations", the server's processing and response time should not exceed 2 seconds. The full loading and interactive time for main user-facing pages (e.g., search results page, booking details page) should not exceed 2 seconds.

4.2.4 Implementation Strategy

To achieve these performance goals, the system design will incorporate "efficient date-range checks and simple caching" to ensure fast availability and quote calculations across the hotels.

4.2.5 Quality Tactics

Usability: The system is based on self service. Customers can complete booking, check in, and check out without staff. The system sends email or SMS when an order status changes. Customers can cancel after booking. The system keeps the order for several minutes so the customer can resume the payment process after leaving the page.

Availability: High availability is required. Use clustering and failover so the service continues when one node stops. After a customer completes booking, lock the room to avoid double booking. Save all orders in the database so we can recover the state after a failure.

Reliability: Allow short interruptions and recover fast through redundant instances and failure detection. When a part fails, the system should keep working with a possible reduction in operating level.

Modifiability and Maintainability: Administrators should be able to identify, repair, or suppress faults and restore normal operation. Use modular and structured programming and design. The architecture should allow independent updates of components without affecting system stability.

Extensibility: The system can include more hotels by recording hotel and room data in the database. We use Java Spring Boot, so new functions can be added as services and released to production after tests without affecting existing services.

Portability: The backend API service is designed for high portability. The Java based backend can run on Linux, Windows, and macOS and on cloud platforms that support the Java virtual machine.

Security: Use a third party identity provider such as Google Sign In. Enforce access control based on user roles. After login the customer holds a credential that expires after several hours. Validate inputs on the client and the server to avoid SQL injection. Logs should mask sensitive data such as the card number, email, and mobile phone, and show only the last four digits.

Testability: The structure follows MVC and supports separation of concerns. Each layer can be tested on its own. Use JUnit for unit and integration tests.

4.3 GUI Prototypes

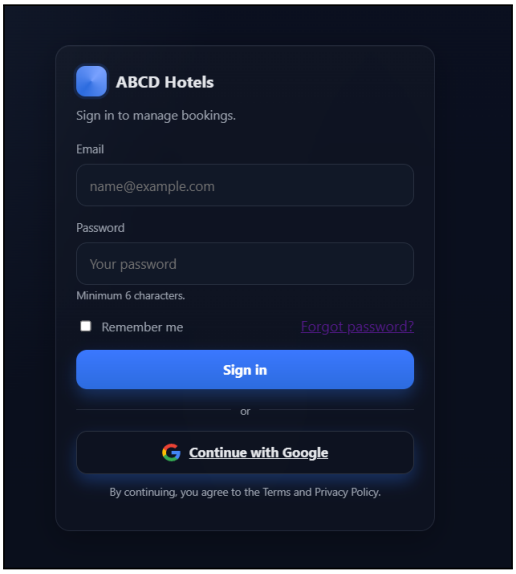


Figure 3: Login Page

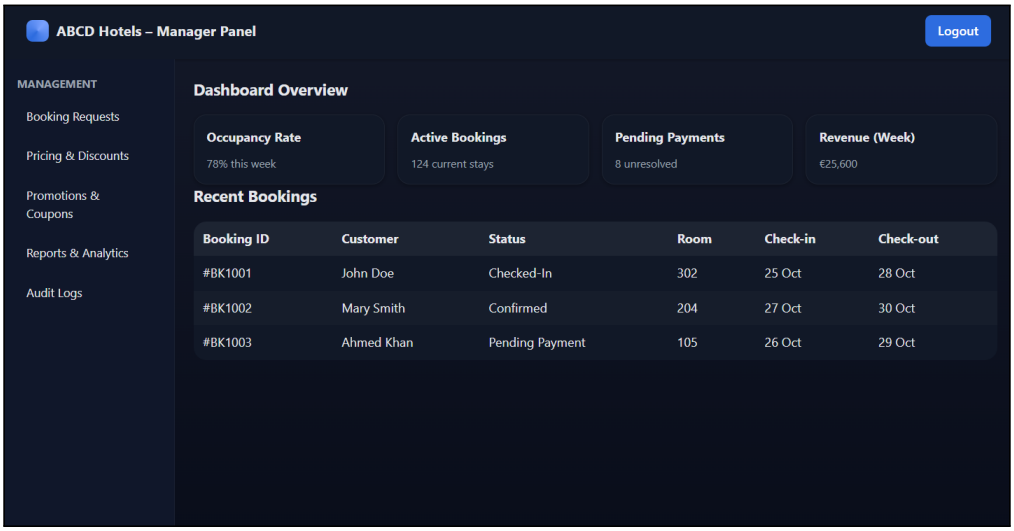


Figure 4: Manager Dashboard

5 System Architecture

5.1 Overview

The system is a monolithic web application that follows a request-response. It uses Spring MVC to separate user interface, request handling, and business rules. Core data is stored in PostgreSQL. Redis supports caching, rate limiting, lightweight publish-subscribe events, and a short-lived key and lock during booking sessions. Java MailSender delivers email notifications. Spring Security and Google sign-in provide authentication. Important data is stored in encrypted form and hidden in system logs. The audit log is encrypted and append-only to support accountability

5.1.1 Architectural Pattern 1: Model-View-Controller (MVC)

The MVC pattern splits an application into three parts. A client sends a request, the controller handles the request and asks the model to do the work, and returns the result to the view. This separation makes change easier. The model is the central part that manages data and business rules. It loads and updates state through repositories, enforces invariants, and performs calculations such as pricing or availability checks. It exposes simple operations and returns results that are independent from any user interface. Presentation logic is not included here. The controller is the entry point for requests and the link between user and model. It reads and validates input, calls the correct model operation, and handles errors. After the model finishes, the controller selects a view and passes the data to it. The view is responsible for presentation only. It renders the data as HTML or JSON, adds response headers, and sets clear status codes. It does not run business rules or calculations, its role is to show information in a form the client can use.

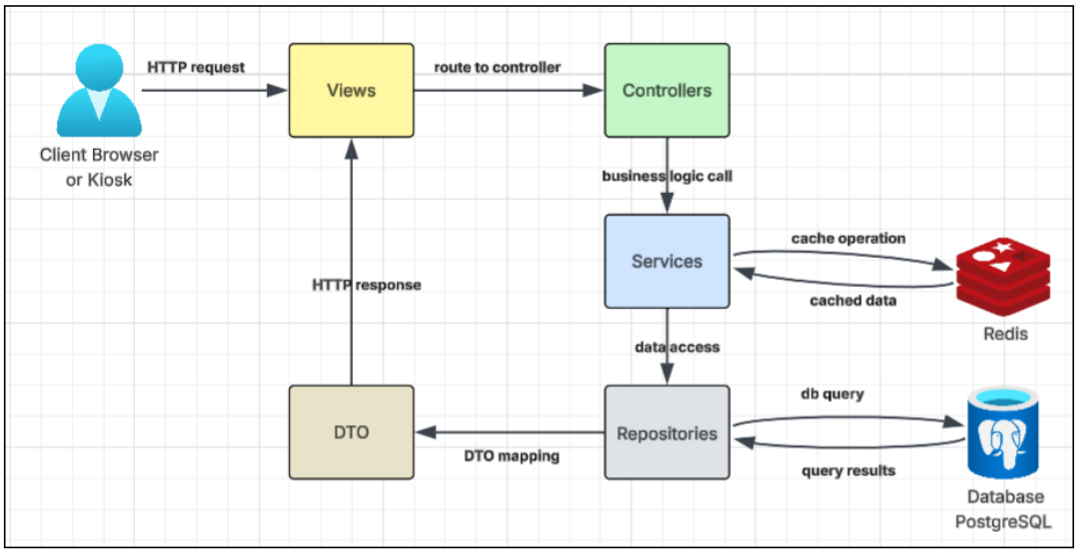


Figure 5: MVC request flow diagram for our case

5.1.2 Architectural Pattern 2: Publish-Subscribe with Redis

Publish-subscribe is a style where parts of the system exchange events through a broker. After an important action one part sends an event, the broker forwards it, and any part that asked for that type of event reacts. This reduces direct links and keeps the main request response path fast. It is used for internal notifications only. Business data stays in PostgreSQL. Every important action is also written to the encrypted

audit log. A channel is a named topic in Redis. Services subscribe to a channel during startup. This is a one time setup step. When work is complete the publisher sends a small event to the channel and continues without waiting. Messages are small and short lived. The broker does not replace the database or the audit log.

Figure 3 shows the basic flow. SendMailService subscribes to the channel at startup. BookingService finishes a task such as confirming a booking and publishes an event to the channel. The channel broadcasts the event to all subscribers. SendMailService receives the event and sends an email through Java MailSender. Other subscribers can run small tasks such as clearing related cache in Redis. These tasks run outside the user request so they do not add delay. In practice a service saves its changes, then publishes an event to a Redis channel. Redis delivers the event to all subscribers. Each subscriber completes a small task. The system stays consistent because PostgreSQL and the encrypted audit log hold the source. This pattern supports fast responses and clear separation inside the monolith.

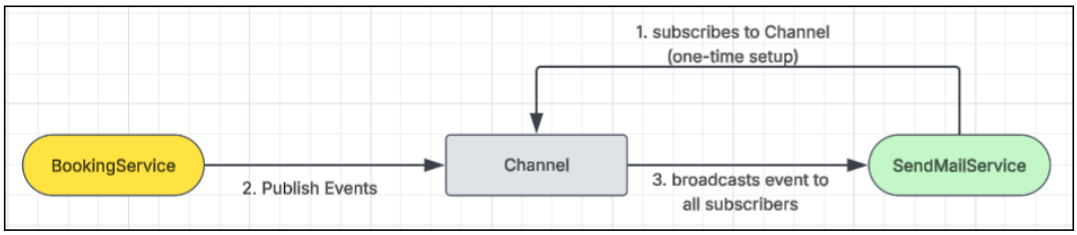


Figure 6: publish–subscribe flow diagram for our case

5.2 Common Services and Rules

- The system stores frequently read data in Redis, such as availability windows and price components. Cache entries use short TTL values, and when bookings change, entries are invalidated by room and date range.
- Redis enforces rate limits with per-user or per-IP counters on sensitive endpoints like search and price quote. When a limit is reached, the API returns an error.
- When a user selects a room, the application creates a short-time lock in Redis to prevent double booking. The lock expires automatically to avoid process deadlocks.
- Spring Security provides authentication and access control. The system supports username–password and Google sign-in. Roles for Guest and Manager are enforced, and logs hide sensitive fields.
- Personal data is encrypted, and all network traffic is protected with TLS. The audit log is also encrypted and append-only, with each entry recording time, actor, and action.
- Java MailSender sends booking confirmations and status updates through SMTP.

5.3 Technology Pipeline

- **Spring MVC:** the main web framework that implements the Model–View–Controller pattern on the Java Virtual Machine.
- **Nginx:** the web server and reverse proxy. It terminates TLS, serves static files efficiently, and can enforce simple edge rate limits and request size limits.
- **PostgreSQL:** the database that stores reservations, users, rooms, prices, and audit entries. It uses clear schemas with date range indexes to support fast availability checks and reporting.
- **Redis:** the in-memory data store. It provides caching for frequently accessed data, rate limiting for sensitive endpoints, short-lived locks to protect bookings, and publish–subscribe channels for events.

- **Postman:** the API testing tool. It builds and runs HTTP requests, stores test collections, and helps document and verify the system's APIs.
- **JUnit 5:** automated unit and integration testing for controllers, services, and repositories, executed locally and in CI.

5.4 High-level Package Diagram

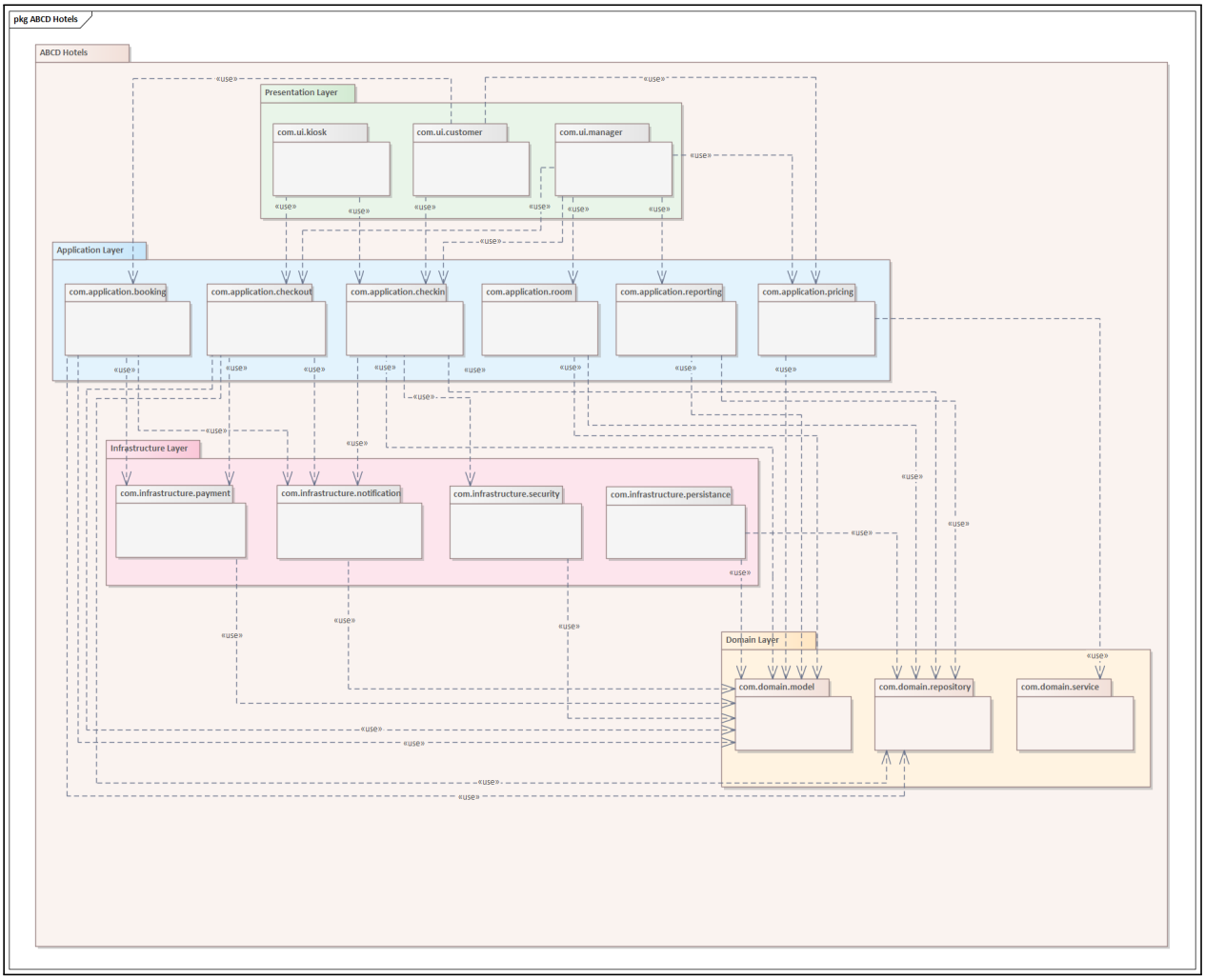


Figure 7: Package Diagram

6 Conceptual Design

6.1 List of Candidate Objects

- Hotel
- Room
- Customer
- Manager
- RoomType
- RoomAvailability
- Order
- Price
- IPaymentProvider
- INotification
- IPricingRule
- BookingService
- CheckInService
- ReportingService
- PricingService

6.2 Sketch of Class Diagram

6.2.1 Analysis Class Diagram – Core Model

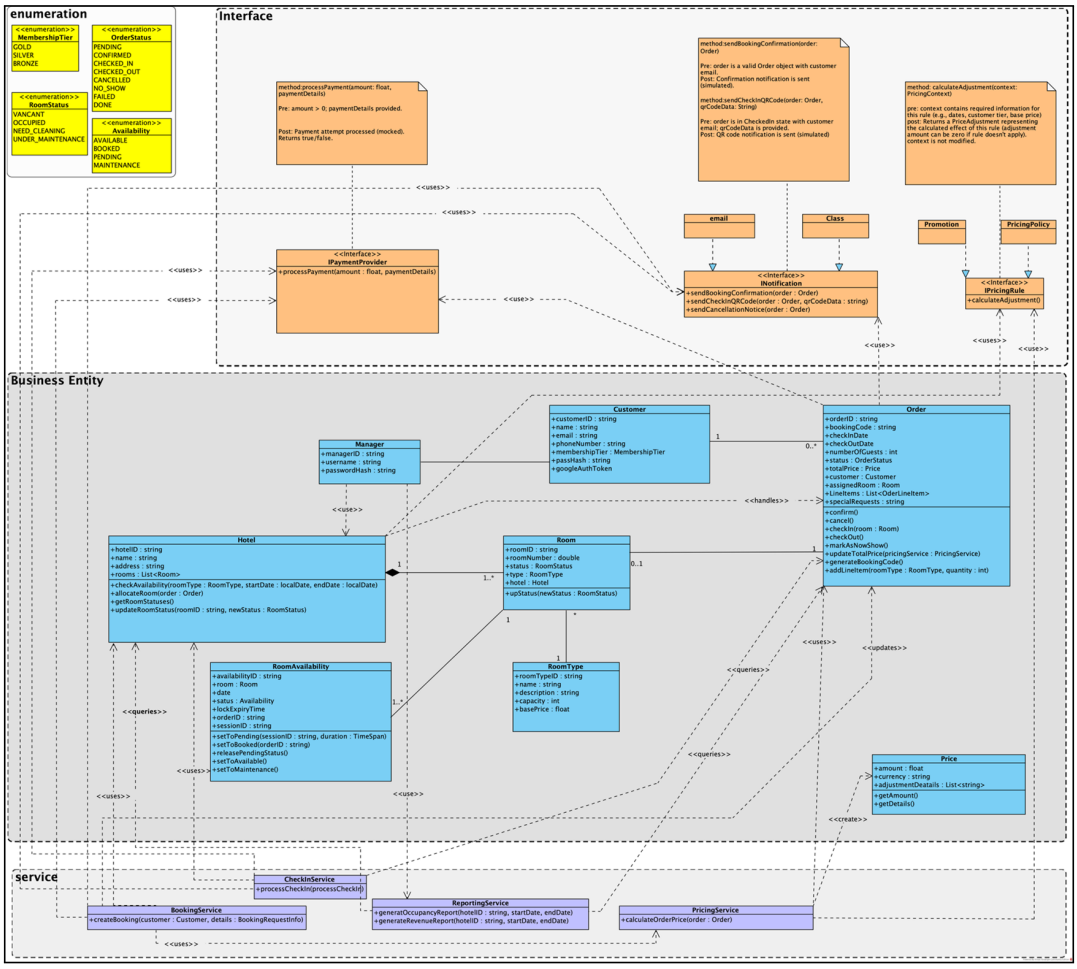


Figure 8: Analysis Class Diagram – Core Model

6.2.2 Analysis Diagram – Create Booking Flow

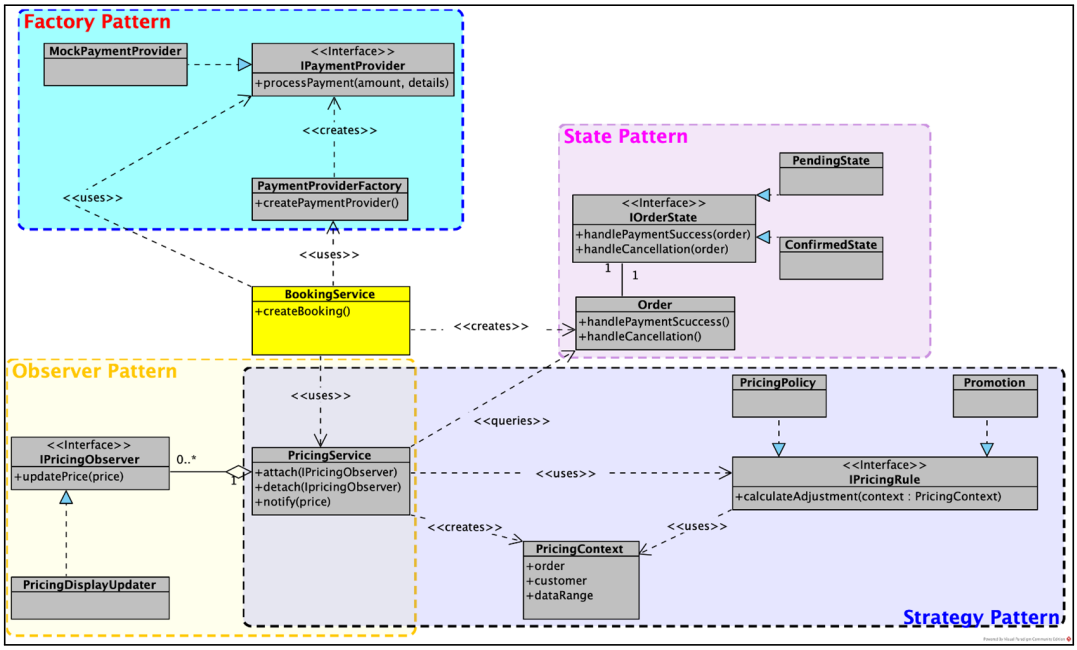


Figure 9: Analysis Diagram – Create Booking Flow

6.2.3 Analysis Diagram – Check-in Flow

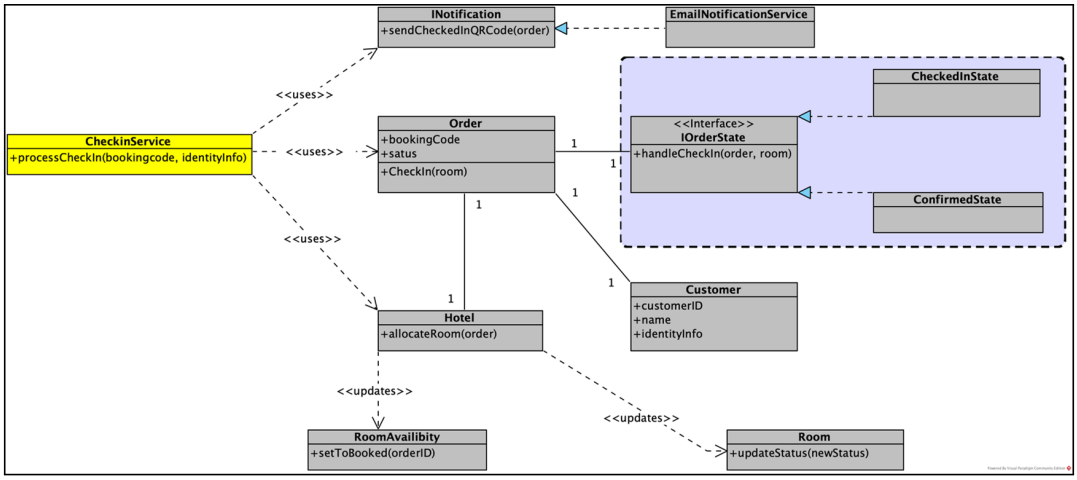


Figure 10: Analysis Diagram – Check-in Flow

6.3 Sequence Diagrams

6.3.1 Searching Room

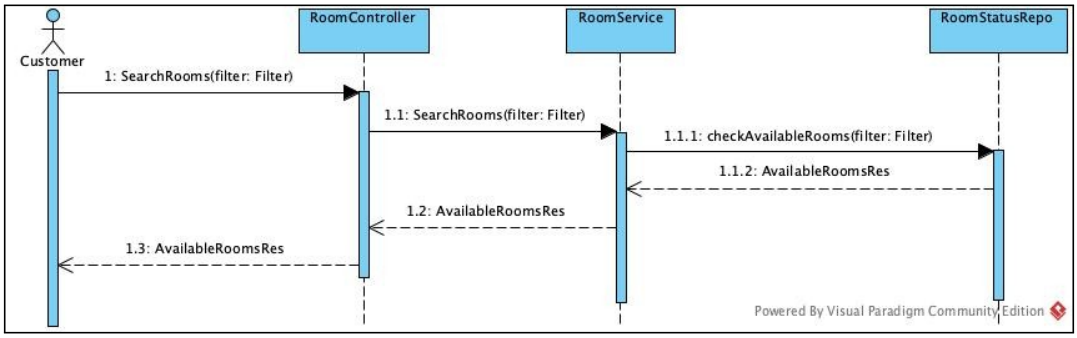


Figure 11: Searching Room Sequence Diagram

6.3.2 Booking Room

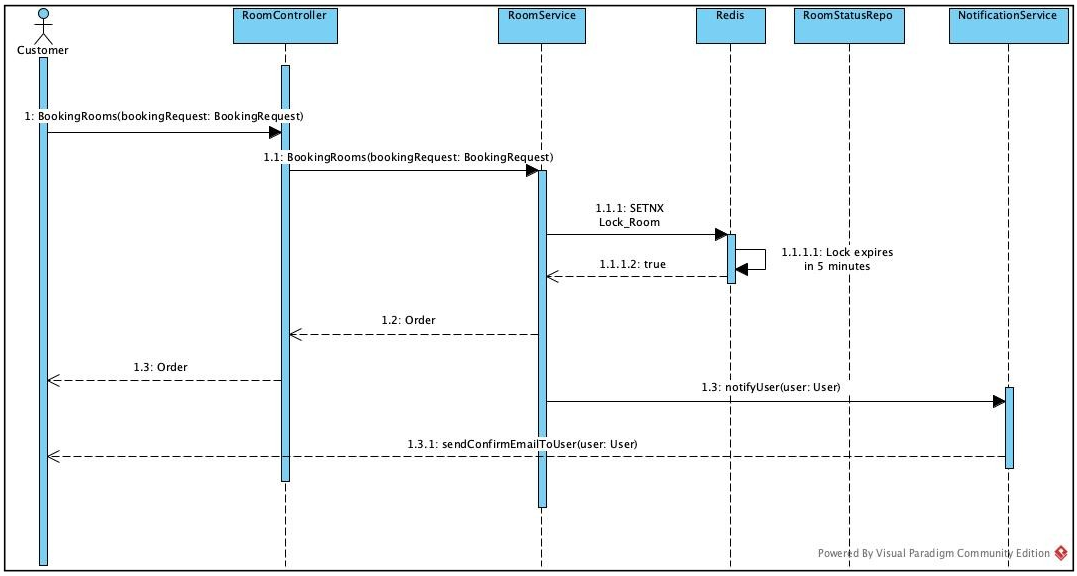


Figure 12: Booking Room Sequence Diagram

6.3.3 Payment

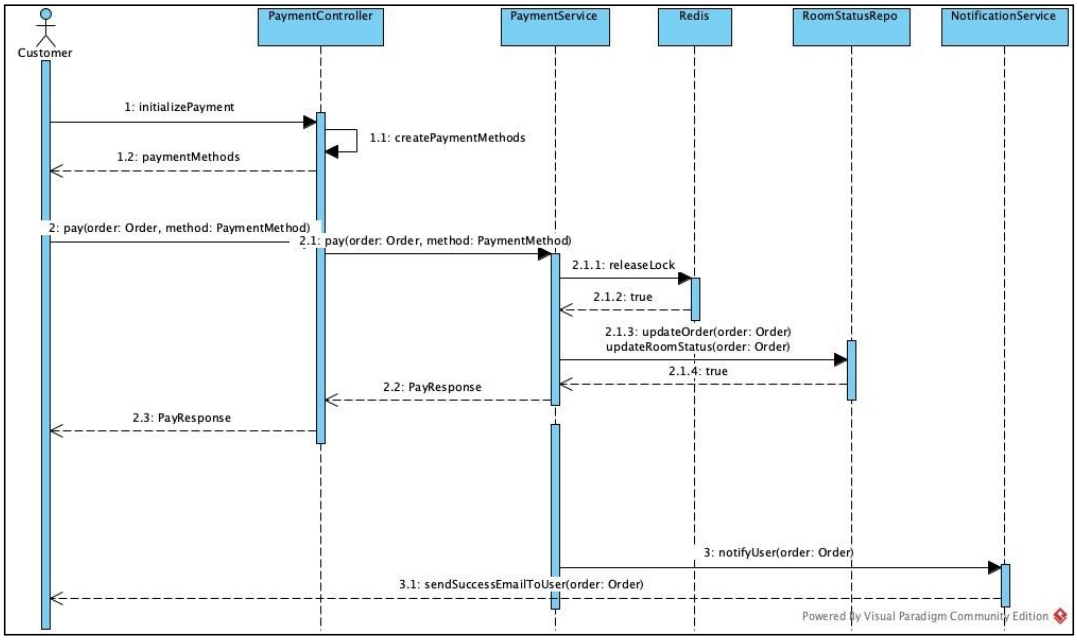


Figure 13: Payment Sequence Diagram

6.4 State Chart Diagram

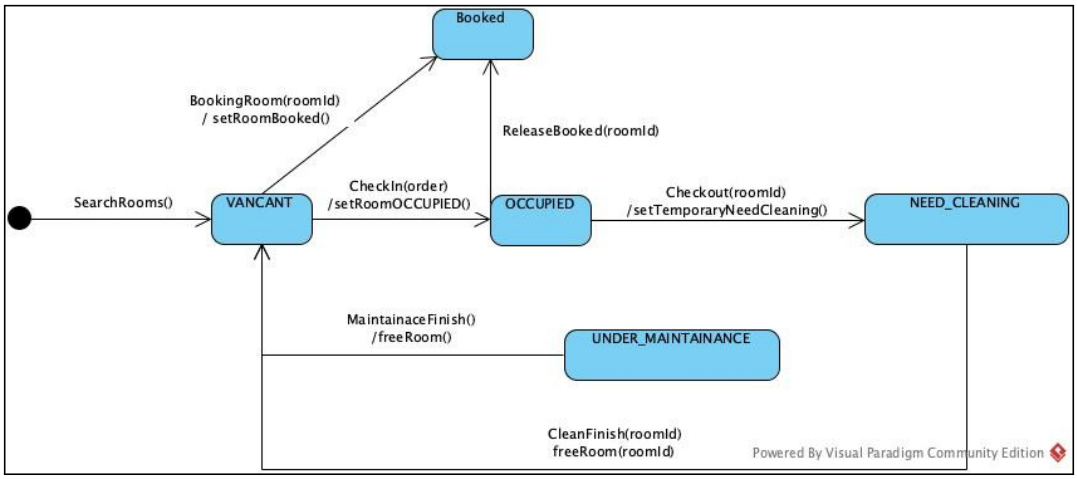


Figure 14: State Chart Diagram

6.5 Entity Relationship Diagram with Cardinality

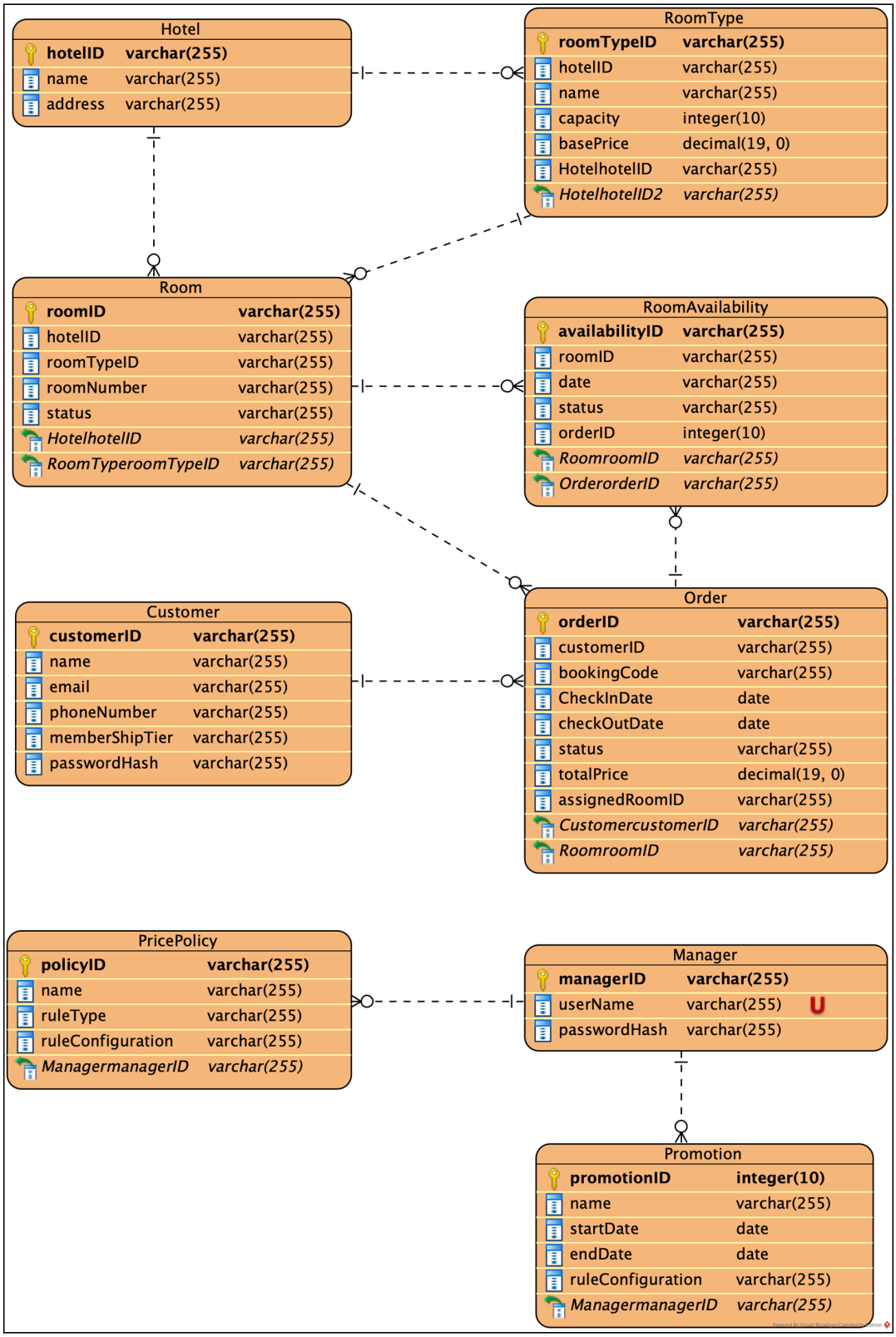


Figure 15: Entity Relationship Diagram with Cardinality

7 Transparency And Traceability

7.1 Classes Per Package

We analysed the Java sources under `src/main/java` and `src/test/java`. The codebase contains **157** classes across **24** packages. Most classes are concentrated in `dto`, `service`, `controllers`, and `base.pricing` (74 classes in total). The remaining classes are spread across smaller packages.

Table 5: Classes per package (summary counts)

Package	Class count
com.example.hotelreservationsystem.dto	26
com.example.hotelreservationsystem.service	19
com.example.hotelreservationsystem.controllers	17
com.example.hotelreservationsystem.base.pricing	12
com.example.hotelreservationsystem.entity	7
com.example.hotelreservationsystem.enums	7
com.example.hotelreservationsystem.service.roomstate	7
com.example.hotelreservationsystem.repository	6
com.example.hotelreservationsystem.security	6
com.example.hotelreservationsystem.events.observers	5
com.example.hotelreservationsystem.pattern	5
com.example.hotelreservationsystem.base.notification	4
com.example.hotelreservationsystem.base.payment	4
com.example.hotelreservationsystem.base.payment.observer	4
com.example.hotelreservationsystem.events	4
com.example.hotelreservationsystem.exception	4
com.example.hotelreservationsystem.payment	4
com.example.hotelreservationsystem	3
com.example.hotelreservationsystem.base.payment.decorator	3
com.example.hotelreservationsystem.config	3
com.example.hotelreservationsystem.pattern.interceptors	3
com.example.hotelreservationsystem.aop	2
com.example.hotelreservationsystem.converter	1
com.example.hotelreservationsystem.integration	1
Total	**157**

7.2 LOC Per Author

We used `git blame` to estimate how many lines are currently attributed to each contributor in the repository. Table 6 shows the main contributors by blame-based LOC.

Table 6: LOC per author

Author	**Blame LOC**
Tan Le	3314
Dechao Zhan	1964
Ismail Bahlaoui	1255
Wenkai zhu	1166

8 Implementation

8.1 Architectural Patterns

8.1.1 MVC Architectural Pattern

The Model-View-Controller (MVC) architectural pattern is employed in the hotel reservation system to separate concerns and improve maintainability.
The application is organised into three main components:

- **Model**: Represents the domain data and business logic of the application. It includes core entities such as *Room*, *Booking*, *Customer*, and *Order*.
- **View**: Responsible for rendering the user interface and presenting data to users. It receives data updates from the Controller and displays the current system state.
- **Controller**: Acts as an intermediary between the Model and the View. It handles user requests, invokes the appropriate business logic, and updates the View accordingly.

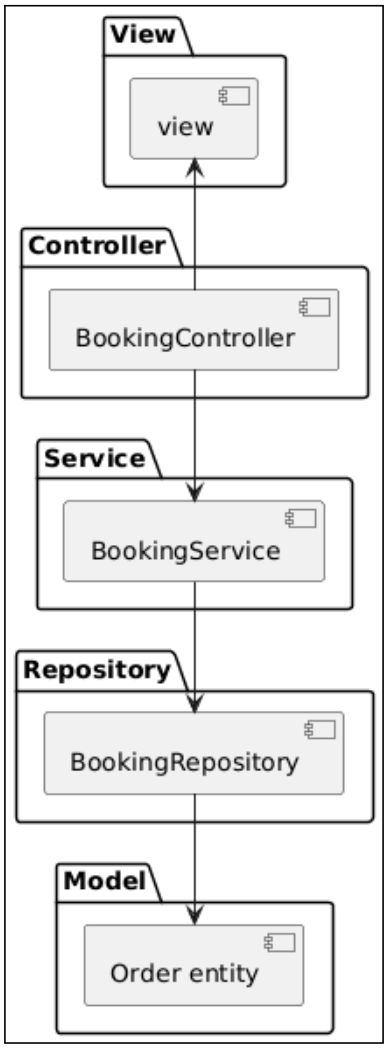


Figure 16: Model-View-Controller (MVC) architectural pattern

8.1.2 Layered Architectural Pattern

Our application is structured into multiple layers, including Presentation, Business Logic, and Data Access layers. All components are deployed together as a single unit, with clear separation of concerns between layers. This separation of concerns enhances maintainability and scalability.
Our package layout includes:

- `com.example.hotelreservationsystem.controllers`
- `com.example.hotelreservationsystem.service`
- `com.example.hotelreservationsystem.dto`
- `com.example.hotelreservationsystem.repository`
- `com.example.hotelreservationsystem.entity`

Presentation Layer: This layer handles user interactions and is responsible for rendering the user interface. It communicates with the Business Logic Layer to process user requests.
Business Logic Layer: This layer contains the core functionality of the application, including business rules and logic.
Repository Layer: This layer manages data access and persistence, interacting with the database to perform CRUD operations.

8.1.3 Interceptor Architectural Pattern

The Interceptor Design Pattern provides a way to intercept and process requests or responses before they reach their intended destination. It allows for cross-cutting concerns such as logging, authentication, and validation to be handled in a modular way.
In our application, we use the Interceptor Design Pattern to implement request logging and authentication.

- `Interceptor` interface defines the contract for interceptors, and concrete interceptor classes implement specific logic.
- `PureAuthInterceptor`, `PureAuthenticationInterceptor`, and `PurePerformanceInterceptor` are concrete implementations of the `Interceptor` interface, each providing its own logic for processing requests.
- `InterceptorChain` manages a list of interceptors and executes them in order.
- `InerceptorManager` helps creating the interceptor chain.
- `AuthenticationController` integrates the interceptor chain to process incoming requests. For example, in `login` and `logout` methods.

```
1  package com.example.hotelreservationsystem.pattern;
2
3  public interface Interceptor {
4
5    boolean before(RequestContext context);
6
7    void after(RequestContext context);
8
9    void afterCompletion(RequestContext context, Exception exception);
10
11   default String getName() {
```

```
    return this.getClass().getSimpleName();
  }

  default int getOrder() {
    return 100;
  }
}
```

Listing 1: Interceptor interface

```
package com.example.hotelreservationsystem.pattern.interceptors;

import com.example.hotelreservationsystem.pattern.Interceptor;
import com.example.hotelreservationsystem.pattern.RequestContext;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Component;

@Slf4j
@Component
public class PureAuthenticationInterceptor implements Interceptor {

  private static final String START_TIME_KEY = "auth.startTime";

  @Override
  public boolean before(RequestContext context) {
    String uri = context.getRequestURI();
    String method = context.getMethod();

    // Store start time
    context.setAttribute(START_TIME_KEY, System.currentTimeMillis());

    log.info("[PURE AUTH] BEFORE - {} {} from IP: {}",
    method, uri, context.getClientIP());

    // Validate based on endpoint
    if (uri.contains("/login")) {
      return validateLogin(context);
    } else if (uri.contains("/logout")) {
      return validateLogout(context);
    } else if (uri.contains("/register")) {
      log.info("[PURE AUTH] Registration request detected");
    }

    return true; // Continue
  }

  @Override
  public void after(RequestContext context) {
    int status = context.getStatusCode();
    String uri = context.getRequestURI();

```

```
42      log.info("[PURE AUTH] AFTER - {} completed with status: {}", uri,
    status);
43
44      // Log result
45      if (status >= 200 && status < 300) {
46        log.info(" [PURE AUTH] Authentication successful");
47      } else if (status == 401) {
48        log.warn(" [PURE AUTH] Authentication failed - Unauthorized");
49      } else if (status >= 400) {
50        log.warn("[PURE AUTH] Authentication failed - Status: {}", status);
51      }
52    }
53
54    @Override
55    public void afterCompletion(RequestContext context, Exception exception) {
56      Long startTime = context.getAttribute(START_TIME_KEY);
57
58      if (startTime != null) {
59        long duration = System.currentTimeMillis() - startTime;
60        log.info("[PURE AUTH] COMPLETE - {} - Total time: {}ms",
61        context.getRequestURI(), duration);
62      }
63
64      if (exception != null) {
65        log.error("[PURE AUTH] Exception occurred: {}",
    exception.getMessage(), exception);
66      }
67
68      // Cleanup
69      context.removeAttribute(START_TIME_KEY);
70    }
71
72  }
```

Listing 2: PureAuthenticationInterceptor implementation

```
1   package com.example.hotelreservationsystem.pattern;
2
3   import lombok.extern.slf4j.Slf4j;
4
5   import java.util.ArrayList;
6   import java.util.Comparator;
7   import java.util.List;
8   import java.util.stream.Collectors;
9
10  public class InterceptorChain {
11
12    private final List<Interceptor> interceptors;
13    private final List<Interceptor> executedInterceptors;
14
15    public <T> T executeChain(RequestContext context, Operation<T> operation)
```

```
    throws Exception {
      if (!executeBefore(context)) {
        log.warn("Request blocked by interceptor chain");
        throw new InterceptorException("Request blocked by interceptor");
      }

      T result = null;
      Exception caughtException = null;

      try {
        result = operation.execute();
        executeAfter(context);

      } catch (Exception ex) {
        caughtException = ex;
      } finally {
        context.markComplete();
        executeAfterCompletion(context, caughtException);
      }

      if (caughtException != null) {
        throw caughtException;
      }

      return result;
    }
}
```

Listing 3: InterceptorChain execution logic

```
package com.example.hotelreservationsystem.controllers;

public class AuthenticationController {

  @PostMapping("/login")
  public ResponseEntity<LoginResponse> login(
  @RequestBody @Validated LoginRequest request,
  HttpServletRequest httpRequest,
  HttpServletResponse httpResponse
  ) {
    RequestContext context = new RequestContext(httpRequest, httpResponse);
    InterceptorChain chain = interceptorManager.createAuthChain();

    try {
      LoginResponse response = chain.executeChain(context, () -> {
        return authenticationService.login(request);
      });

      return ResponseEntity.ok(response);

    } catch (Exception ex) {
```

```
22          context.setStatusCode(401);
23          return ResponseEntity.status(401).build();
24        }
25      }
26
27  }
```

Listing 4: AuthenticationController using interceptor chain

8.1.4 Built-in Interceptor

In addition to the custom interceptor chain, the system includes a built-in interceptor implemented using Spring AOP. This interceptor is enabled through a marker annotation, `@BuiltInInterceptor`, which can be placed on controller methods or on a controller class. When the annotation is present, Spring executes aspect logic around the matched controller method.

The interceptor is implemented as an `@Aspect` named `BuiltInAuthInterceptor`. It uses an `@Around` advice to wrap controller execution. Before the controller method runs, it reads request context data such as URI, HTTP method, and client IP (when the current request is available). It then calls a validation hook, which is currently a no-op.

For room-related endpoints (URIs containing `/api/rooms`), it creates an audit event, stores it in an in-memory list, and logs the final audit result after the request completes. For non-room endpoints, the request still receives logging and performance recording, but no audit event is created.

After the controller returns, the interceptor logs the HTTP status and records a performance metric that includes execution time. The metrics list is capped (maximum 1000 entries) to limit in-memory growth. The interceptor also logs warnings when execution time exceeds 500ms or 1000ms. If an exception occurs, it logs the error, completes the audit event when applicable, records the performance metric, and then rethrows the exception.

Marker annotation: BuiltInInterceptor This annotation enables Spring AOP interception on controller methods or controller classes.

```
1  @Target({ElementType.METHOD, ElementType.TYPE})
2  @Retention(RetentionPolicy.RUNTIME)
3  public @interface BuiltInInterceptor {
4  }
```

Listing 5: BuiltInInterceptor marker annotation

Aspect: around advice for intercepted endpoints This fragment shows the `@Around` advice that wraps controller execution. It performs request logging, room-endpoint auditing, and performance recording.

```
1  @Around("@annotation(com.example.hotelreservationsystem.aop
2  .BuiltInInterceptor)"
3  + " || @within(com.example.hotelreservationsystem.aop.BuiltInInterceptor)")
4  public Object aroundAuthEndpoints(ProceedingJoinPoint pjp) throws Throwable
     {
5    long start = System.currentTimeMillis();
```

```
    HttpServletRequest request = currentRequest();
    HttpServletResponse response = currentResponse();

    String uri = request != null ? request.getRequestURI() : "N/A";
    String method = request != null ? request.getMethod() : "N/A";
    String clientIp = request != null ? clientIp(request) : UNKNOWN;

    AuditEvent auditEvent = beginAudit(uri, method, clientIp, request);

    log.info("[AOP AUTH] BEFORE - {} {} from IP: {}", method, uri, clientIp);
    validate(request, uri);

    try {
      Object result = pjp.proceed();

      int status = response != null ? response.getStatus() : 200;
      log.info("[AOP AUTH] AFTER - {} completed with status: {}", uri,
  status);

      completeAudit(auditEvent, status, null);
      recordPerformance(uri, method, clientIp, status, start);
      return result;

    } catch (Exception ex) {
      log.error("[AOP AUTH] Exception during {} {}: {}", method, uri,
  ex.getMessage(), ex);

      if (response != null && response.getStatus() == 0) {
        response.setStatus(500);
      }

      int status = response != null ? response.getStatus() : 500;
      completeAudit(auditEvent, status, ex);
      recordPerformance(uri, method, clientIp, status, start);
      throw ex;

    } finally {
      long duration = System.currentTimeMillis() - start;
      log.info("[AOP AUTH] COMPLETE - {} {} took {}ms", method, uri,
  duration);
    }
  }
```

Listing 6: BuiltInAuthInterceptor around advice for @BuiltInInterceptor endpoints

Room endpoint filter and performance thresholds Auditing is applied only to room endpoints, and performance warnings are logged when execution time exceeds defined thresholds.

```
  private boolean isRoomsEndpoint(String uri) {
    return uri != null && uri.contains("/api/rooms");}
```

```
  private static final long WARNING_THRESHOLD = 500; // 500ms
  private static final long SLOW_THRESHOLD = 1000;   // 1 second
```

Listing 7: Room endpoint filter and performance thresholds

8.1.5 Dependency Injection Architectural Pattern

Dependency Injection (DI) is a design pattern used to implement Inversion of Control (IoC) for resolving dependencies. In our application, we utilize DI to manage the dependencies between various components, such as services and repositories. We use Spring Framework's built-in DI capabilities to inject dependencies at runtime, promoting loose coupling and enhancing testability.
BookingService depends on multiple repositories and services, which are injected via the constructor. For example, OrderRepository is injected into BookingService, allowing it to perform database operations related to orders without being tightly coupled to the repository implementation.
A brief example of DI in our application is shown below:

```
@Service
public class BookingService {
  private final OrderRepository orderRepository;
  private final BookingLockService bookingLockService;
  private final RoomRepository roomRepository;
  private final CustomerRepository customerRepository;
  private final NotificationServiceFactory notificationServiceFactory;
  private final RoomService roomService;

  // logic methods
}
```

Listing 8: BookingService with injected dependencies

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
  Optional<Order> findByRoomId(Long roomId);
}
```

Listing 9: OrderRepository interface

8.2 Design Pattern

8.2.1 Observer Design Pattern

Observer Design Pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. For example, `PricingObject` is the subject that holds the current price of a room. `PricingObserver` is the observer interface that defines the reacts to price changes.
Observer and ConcreteObserver implementation:

```
// PricingObserver.java
package com.example.hotelreservationsystem.base.pricing;
```

```

public interface PricingObserver {
  void update(float price);
}
```

Listing 10: PricingObserver interface

```
// OrderPriceObserver.java
package com.example.hotelreservationsystem.base.pricing;

import com.example.hotelreservationsystem.entity.Order;
import com.example.hotelreservationsystem.repository.OrderRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

public class OrderPriceObserver implements PricingObserver {

  private final Order order;
  private final OrderRepository orderRepository;

  @Override
  public void update(float newPrice) {
    // Update order's total price using its Observer method
    order.update(newPrice);

    // Persist the updated order
    orderRepository.save(order);
  }
}
```

Listing 11: OrderPriceObserver implementation

Subject and ConcreteSubject implementation:

```
// PricingObject.java
package com.example.hotelreservationsystem.base.pricing;

public interface PricingObject {
  void attach(Long roomType, PricingObserver observer);

  void detach(Long roomType, PricingObserver observer);

  void notify(Long roomType, Float price);
}
```

Listing 12: PricingObject interface

```
// RoomService.java
public class RoomService implements PricingObject {

  private final ConcurrentHashMap<Long, List<PricingObserver>>
 _dependencies = new ConcurrentHashMap<>();

```

```
    // Long: roomType, Float: price
    private final ConcurrentHashMap<Long, Float> _prices = new
  ConcurrentHashMap<>();

    @Override
    public void attach(Long roomType, PricingObserver observer) {
      _dependencies.computeIfAbsent(roomType, _ -> new
  ArrayList<>()).add(observer);
    }

    @Override
    public void detach(Long roomType, PricingObserver observer) {
      List<PricingObserver> observers = _dependencies.get(roomType);
      if (observers != null) {
        observers.remove(observer);
      }
    }

    @Override
    public synchronized void notify(Long roomType, Float price) {
      List<PricingObserver> observers = _dependencies.get(roomType);
      if (observers != null) {
        for (PricingObserver observer : observers) {
          observer.update(price);
        }
      }
    }

    public void _updatePrice(Long roomType, float price) {
      Float oldPrice = _prices.get(roomType);
      if (oldPrice == null || !oldPrice.equals(price)) {
        _prices.put(roomType, price);
        notify(roomType, price);
      }
    }

    @Transactional
    public RoomType updatePrice(Long roomTypeId, float price) {
      var roomTypeOpt = roomTypeRepository.findById(roomTypeId);
      if (roomTypeOpt.isEmpty()) {
        return null;
      }

      var roomType = roomTypeOpt.get();
      roomType.setPrice(price);
      roomTypeRepository.save(roomType);

      _updatePrice(roomTypeId, price);
      return roomType;
    }
```

```
54
55  }
```

Listing 13: RoomService as ConcreteSubject

When the price of a room type is updated in `RoomService`, it calls the `notify` method to inform all registered observers (e.g., `OrderPriceObserver`) about the price change. The observers then update their state accordingly.

8.2.2 Factory Design Pattern

Factory Design Pattern encapsulates the creation logic in a separate factory class. It helps promote loose coupling, and single responsibility principle.
In our application, `Notification` is an interface representing a notification service. `EmailNotification` and `SMSNotification` are concrete implementations of the `Notification` interface.

`NotificationServiceFactory` class is used to generate different types of notification service based on `NotificationType`, including Email, SMS. This pattern ensures that client code does not need to know the details of how these services are created.

```
1  // Notification.java
2  package com.example.hotelreservationsystem.base.notification;
3
4  public interface Notification<T> {
5    boolean sendNotification(T message);
6  }
```

Listing 14: Notification interface

```
1  // NotificationServiceFactory.java
2  public class NotificationServiceFactory {
3    private final EmailNotification emailNotification;
4    private final SMSNotification smsNotification;
5
6    public NotificationServiceFactory(EmailNotification emailNotification,
     SMSNotification smsNotification) {
7      this.emailNotification = emailNotification;
8      this.smsNotification = smsNotification;
9    }
10
11   @SuppressWarnings("unchecked")
12   public <T> Notification<T> createNotificationService(NotificationType
     type) throws Exception {
13     if (type == NotificationType.EMAIL) {
14       return (Notification<T>) emailNotification;
15     } else if (type == NotificationType.SMS) {
16       return (Notification<T>) smsNotification;
17     }
18     throw new Exception(type + " not yet implement");
19   }
```

```
20  }
```

Listing 15: NotificationServiceFactory implementation

8.2.3 Decorator Design Pattern

The `Decorator Design Pattern` allows behavior to be added to individual objects, either statically or dynamically, without affecting the behavior of other objects from the same class. It promotes `flexibility` and `reusability`, which adheres to the Open-Close Principle.
In our application, we use the Decorator Pattern to enhance the functionality of the Dynamic Price strategy. The `PricingDecorator` class is used to add additional pricing logic on top of the base room price. `PricingComponent` is the interface that defines the method for calculating the price.
We have three concrete decorators: `HolidayPricingDecorator`, `HolidayPricingDecorator`, and `OccupancyPricingDecorator`. Each decorator adds its own pricing logic to the base price calculation. We use a factory `RoomPriceFactory` to create the price components.

```
1  // PricingComponent.java
2  package com.example.hotelreservationsystem.base.pricing;
3
4  public interface PricingComponent {
5    Float calc();
6  }
```

Listing 16: PricingComponent interface

```
1  // PricingDecorator.java
2  package com.example.hotelreservationsystem.base.pricing;
3
4  class PricingDecorator implements PricingComponent {
5    private final PricingComponent pricingComponent;
6    public PricingDecorator(PricingComponent pricingComponent) {
7      this.pricingComponent = pricingComponent;
8    }
9    @Override
10   public Float calc() {
11     return pricingComponent.calc();
12   }
13 }
```

Listing 17: PricingDecorator base class

```
1  // HolidayPricingDecorator.java
2  package com.example.hotelreservationsystem.base.pricing;
3
4  class HolidayPricingDecorator extends PricingDecorator {
5
6    public HolidayPricingDecorator(PricingComponent pricingComponent) {
7      super(pricingComponent);
8    }
9
10   @Override
```

```
    public Float calc() {
      float holidayRate = 1.5f;
      return super.calc() * holidayRate;
    }
  }
```

Listing 18: HolidayPricingDecorator implementation

The below code shows how to use the `RoomPriceFactory` to create a pricing component with multiple tiers, and how it is integrated into the `RoomController` to update room type prices.

```
package com.example.hotelreservationsystem.base.pricing;

@Service
public class RoomPriceFactory {

  public PricingComponent createPriceComponent1(List<RoomPriceTier>
 priceTiers, float price) {
    PricingComponent component = new RoomPricing(price);
    for (RoomPriceTier tier : priceTiers) {
      component = switch (tier) {
        case OCCUPANCY -> new OccupancyPricingDecorator(component, 0.5f);
        case HOLIDAY -> new HolidayPricingDecorator(component);
        default -> component;
      };
    }
    return component;
  }
}

// RoomController.java
public ResponseEntity<RoomType> updateRoomTypePrice(@RequestBody @Valid
 RoomPriceUpdateRequest request) {
  try {
    var tiers = request.getPriceTiers();
    var priceComponent = roomPriceFactory.createPriceComponent1(tiers,
 request.getPrice());
    var price = priceComponent.calc();
    var updatedRoomType = roomService.updatePrice(request.getRoomTypeId(),
 price);
    if (updatedRoomType == null) return ResponseEntity.notFound().build();
    return ResponseEntity.ok(updatedRoomType);
  } catch (Exception e) {
    log.error(e.getMessage(), e);
    return ResponseEntity.internalServerError().build();
  }
}
```

Listing 19: RoomPriceFactory and controller usage

8.2.4 State Design Pattern

The State Design Pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class. It encapsulates state-specific behavior and transitions within state classes, promoting cleaner code and adherence to the Single Responsibility Principle.
In our application, we use the State Design Pattern to manage the different states of reservation lifecycle states. Each state (e.g., Pending, Confirmed, Cancelled, Completed) will implement its own behavior for these methods.

- `ReservationState` interface defines the contract for state transitions, and concrete state classes implement the specific behaviors.
- `ReservationContext` maintains a reference to the current state and delegates state-specific behavior to the current state object.
- `PendingState`, `ConfirmedState`, `CancelledState`, and `CompletedState` are concrete implementations of the `ReservationState` interface, each defining the allowed transitions from that state.

```java
// ReservationState.java
package com.example.hotelreservationsystem.service.roomstate;

public interface ReservationState {

  void confirm(ReservationContext context);

  void cancel(ReservationContext context);

  void complete(ReservationContext context);

  default void stay(ReservationContext context, OrderStatus status) {
    context.transitionTo(status);
  }
}
```

Listing 20: ReservationState interface

```java
// ReservationContext.java
package com.example.hotelreservationsystem.service.roomstate;

public class ReservationContext {

  private final Order order;

  public ReservationContext(Order order) {
    this.order = order;
  }

  public void confirm() {
    getState(order.getOrderStatus()).confirm(this);
  }

```

```
    public void cancel() {
      getState(order.getOrderStatus()).cancel(this);
    }

    public void complete() {
      getState(order.getOrderStatus()).complete(this);
    }

    void transitionTo(OrderStatus status) {
      order.setOrderStatus(status);
    }

    private ReservationState getState(OrderStatus status) {
      return switch (status) {
        case PENDING -> new PendingState();
        case CONFIRMED -> new ConfirmedState();
        case CANCELLED -> new CancelledState();
        case COMPLETED -> new CompletedState();
      };
    }
  }
```

Listing 21: ReservationContext implementation

```
  // PendingState.java
  package com.example.hotelreservationsystem.service.roomstate;

  public class PendingState implements ReservationState {

    @Override
    public void confirm(ReservationContext context) {
      context.transitionTo(OrderStatus.CONFIRMED);
    }

    @Override
    public void cancel(ReservationContext context) {
      context.transitionTo(OrderStatus.CANCELLED);
    }

    @Override
    public void complete(ReservationContext context) {
      // Not allowed; keep pending
    }
  }
```

Listing 22: PendingState implementation

8.2.5 Strategy Design Pattern

The Strategy Design Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It allows the algorithm to vary independently of the clients.

In our application, we use the Strategy Design Pattern to implement different payment methods.

- `PaymentStrategy` interface defines the contract for payment processing, and concrete strategy classes implement specific payment methods such as Credit Card, PayPal, and Bank Transfer.
- `CreditCardPayment` and `PayPalPayment` are concrete implementations of the `PaymentStrategy` interface, each providing its own logic for processing payments.
- `PaymentFactory` delegates payment processing to the selected strategy.

```java
// PaymentStrategy.java
package com.example.hotelreservationsystem.base.payment;

public interface PaymentStrategy {
  PaymentResponse pay(PaymentRequest request);
}
```

Listing 23: PaymentStrategy interface

```java
// CreditCardPaymentStrategy.java
package com.example.hotelreservationsystem.base.payment;

@Service
public class CreditCardPaymentStrategy implements PaymentStrategy {

  @Override
  public PaymentResponse pay(PaymentRequest request) {
    return PaymentResponse.builder()
    .orderId(request.getOrderId())
    .status("SUCCESS")
    .transactionId(transactionId)
    .message("Credit Card payment processed successfully.")
    .build();
  }
}
```

Listing 24: CreditCardPaymentStrategy implementation

```java
// PaymentFactory.java
package com.example.hotelreservationsystem.base.payment;

@Component
@RequiredArgsConstructor
public class PaymentFactory {

  public PaymentStrategy getStrategy(PaymentType paymentType) {

    PaymentStrategy rawStrategy = switch (paymentType) {
      case CREDIT_CARD -> new CreditCardPaymentStrategy();
      case PAYPAL -> new PayPalPaymentStrategy();
      default -> throw new IllegalArgumentException("Invalid payment type");
    };
```

```
15
16      return rawStrategy;
17    }
18  }
```

Listing 25: PaymentFactory implementation

8.3 Unit Test

We use JUnit and Mockito for unit testing our services and controllers. In this application, we have implemented approximately **30** unit tests file covering various components. Below is an example of a unit test for the `AuthenticationServiceTest` class.

```
1   // AuthenticationServiceTest.java
2   package com.example.hotelreservationsystem.service;
3
4   @BeforeEach
5   void setUp() {
6     mockCustomer = new Customer();
7     mockCustomer.setId(1L);
8     mockCustomer.setName("Test User");
9     mockCustomer.setEmail("test@example.com");
10    mockCustomer.setPasswordHash("hashedPassword");
11    mockCustomer.setPhoneNumber("1234567890");
12    mockCustomer.setRole(Role.USER);
13    mockCustomer.setMembershipTier(MembershipTier.BRONZE);
14    mockCustomer.setEnabled(true);
15
16    registerRequest = new RegisterRequest("Test User", "test@example.com",
      "password123", "1234567890");
17    loginRequest = new LoginRequest("test@example.com", "password123");
18  }
19
20  @Test
21  void testLoadUserByUsername_Success() {
22    when(customerRepository.findByEmail("test@example.com"))
23    .thenReturn(Optional.of(mockCustomer));
24
25    var userDetails =
      userDetailsService.loadUserByUsername("test@example.com");
26
27    assertNotNull(userDetails);
28    assertEquals("test@example.com", userDetails.getUsername());
29    verify(customerRepository, times(1)).findByEmail("test@example.com");
30  }
31
32  @Test
33  void testLoadUserByUsername_NotFound() {
34    when(customerRepository.findByEmail("test@example.com"))
35    .thenReturn(Optional.empty());
```

```
36
37      assertThrows(UsernameNotFoundException.class, () ->
        userDetailsService.loadUserByUsername("test@example.com"));
38  }
```

Listing 26: AuthenticationServiceTest unit test example

8.4 Version Control

We use Git for version control and host our repository on GitHub. The repository is organized with branches for different features and bug fixes. We follow a branching strategy that includes a main branch for stable releases, a develop branch for ongoing development, and feature branches for specific tasks.
When we complete a feature or fix a bug, we create a pull request to merge the changes into the develop branch. This allows for code review and ensures that the main branch remains stable.
We also include code reviews to help maintain code quality and share knowledge among team members. Each pull request is reviewed by at least one other team member before being merged. Now, we have **25** branches in total, and more than **144** commits.

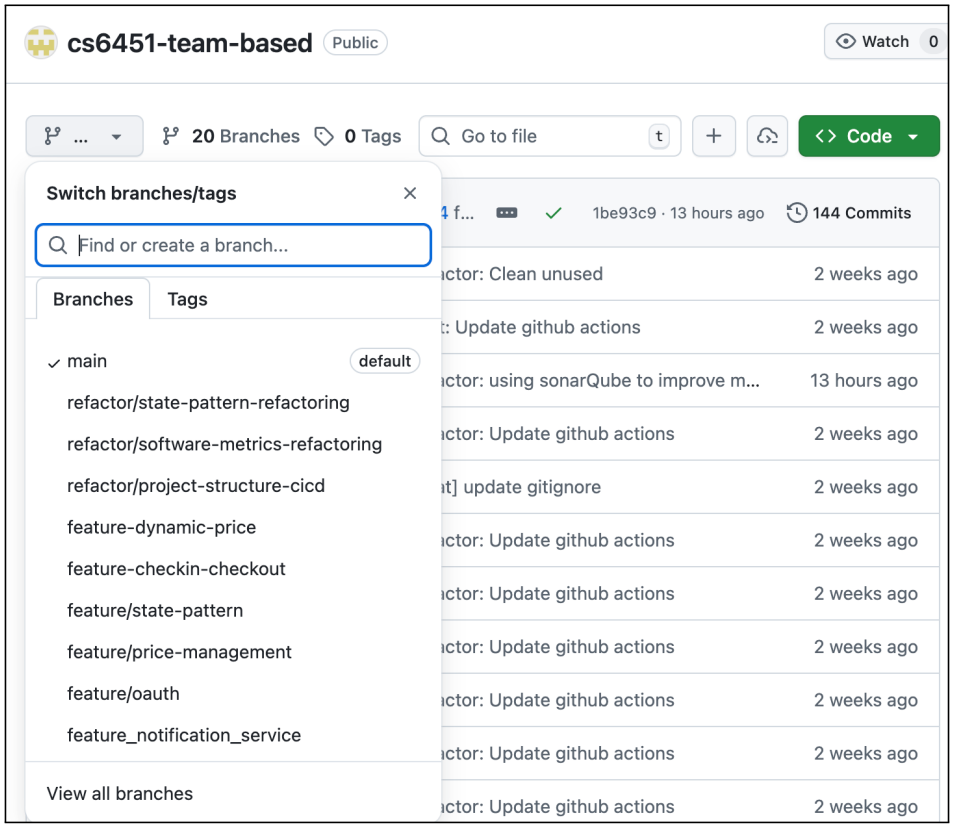


Figure 17: GitHub repository branches example

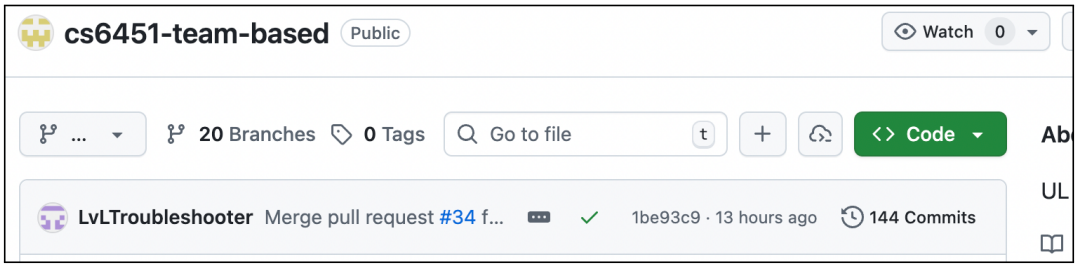


Figure 18: Closed pull requests and merge example

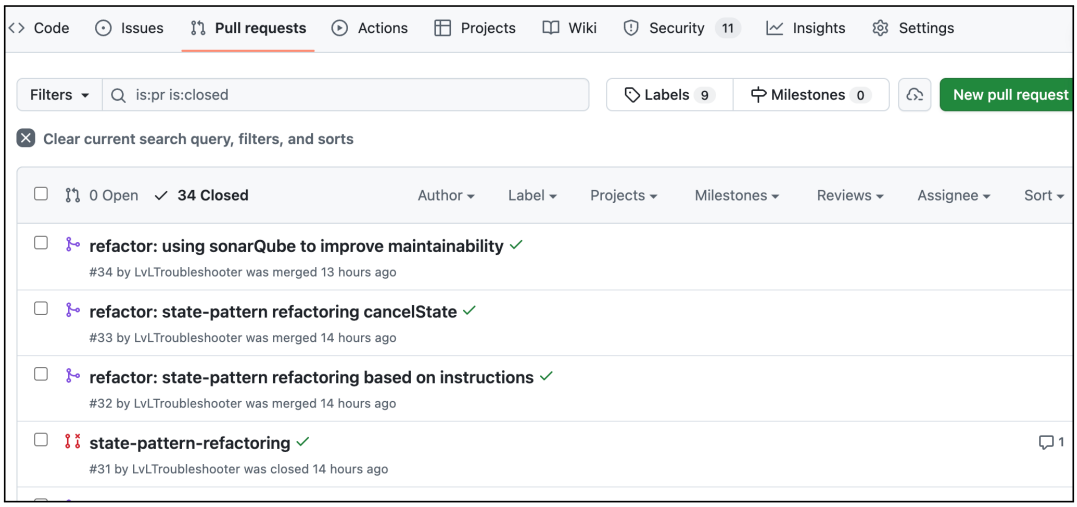


Figure 19: Repository activity and commit example

9 Added Value

9.1 Continuous Integration and Continuous Delivery

9.1.1 Docker

Docker uses Linux kernel features like Cgroups and namespaces to isolate processes. This lets multiple applications run independently on the same infrastructure while maintaining security. Docker provides an image-based deployment model. You package your application with all dependencies into a container image. This image runs consistently across any environment. Docker Compose defines and runs multi-container applications. You configure all services in a single YAML file. One command starts everything. Compose works across production, staging, development, testing, and CI workflows. You get commands to manage the complete application lifecycle: start, stop, rebuild services, view service status, stream logs, and run one-off commands. We use Docker and Docker Compose for portability. One configuration runs on any operating system. Teammates spin up the complete environment with a single command. No manual database installation. No Redis setup. No environment variable conflicts.

9.1.2 GitHub Actions

GitHub Actions automates the build, test, and deployment pipeline. Every push to the repository triggers a workflow. The workflow runs unit tests, integration tests, and builds Docker images. Failed tests block the merge. This catches bugs before they reach production. We configured Actions to deploy to staging after successful builds. The workflow pushes Docker images to a registry, connects to the staging server, pulls the latest images, and restarts services. This eliminates manual deployment steps and reduces human error. The combination of Docker and GitHub Actions creates a professional development workflow. Code changes move from development to staging automatically. The team sees integration issues within minutes of pushing code. This setup mirrors practices at software companies and prepares us for industry work.

9.2 Pattern-based payment module

9.2.1 Description

In the payment module, validation is implemented as a decorator around PaymentStrategy instead of being built into the strategies or the service itself. This has several advantages. The core payment strategies (such as CreditCardPaymentStrategy and PayPalPaymentStrategy) stay small and focus only on communication with the external provider. PaymentValidationDecorator takes care of checking business rules before the payment is executed. The same decorator can be reused for all strategies, so new payment methods automatically reuse the existing checks. The decorator chain is created in PaymentFactory, which means the order and combination of decorators can be changed without modifying controllers or PaymentService.

The validation decorator collects several pre-payment checks that would otherwise be spread across the code. Typical rules include: the order must exist, the status must still be PENDING, the order must belong to the current customer, and the requested amount must match the stored totalPrice. If any of these checks fails, the decorator throws a domain-specific exception and does not call the wrapped strategy. By keeping these checks in one place, all payment flows follow the same rules.

Using decorators for validation improves maintainability and extensibility. When new rules are needed, for

example an extra fraud check or a time restriction on late payments, we can update PaymentValidationDecorator or add a new decorator class and insert it into the chain inside PaymentFactory. Existing strategies and PaymentService do not need to change, which follows the open-closed principle. The structure also makes testing easier: each decorator can be tested on its own, and the full chain can be tested via the PaymentStrategy interface.

9.2.2 Implementation

Field injection of PaymentFactory and List<PaymentObserver> This snippet shows how PaymentService depends only on the abstract PaymentFactory and a collection of PaymentObserver, keeping payment algorithms and side effects loosely coupled to the service.

```
private final List<PaymentObserver> observers;

@Contract(pure = true)
public PaymentService(PaymentFactory paymentFactory, List<PaymentObserver>
 observers) {
  this.paymentFactory = paymentFactory;
  this.observers = observers;
}
```

Listing 27: Field injection of PaymentFactory and PaymentObserver list

ExecutePayment Method This code fragment illustrates the main payment workflow, where PaymentService retrieves the appropriate PaymentStrategy from the factory, executes it, and returns a DTO-based PaymentResponse.

```
public PaymentResponse executePayment(Long orderId, PaymentType
 paymentType) {

  // 1. Prepare Request
  PaymentRequest request = new PaymentRequest();
  request.setOrderId(orderId);
  request.setPaymentType(paymentType);

  // 2. Factory + Decorator: Get the strategy
  PaymentStrategy strategy = paymentFactory.getStrategy(paymentType);

  // 3. Execute (Follow Validation -> retrying -> Real Payment)
  PaymentResponse response = strategy.pay(request);

  // 4. Observer Pattern: Notify all listeners if successful
  if ("SUCCESS".equals(response.getStatus())) {
    notifyObservers(orderId, response);
  }

  return response;
}
```

Listing 28: ExecutePayment method in PaymentService

NotifyObservers methods This snippet demonstrates how PaymentService iterates over all registered PaymentObserver instances after a successful payment to trigger notification, audit, and status-update logic without hard-coding these concerns.

```
private void notifyObservers(Long orderId, PaymentResponse response) {

  log.info(">>> Notifying {} observers...", observers.size());

  for (PaymentObserver observer : observers) {
    try {
      observer.onPaymentSuccess(orderId, response);
    } catch (Exception e) {
      log.error("Observer failed: {}", e.getMessage());
    }
  }
}
```

Listing 29: notifyObservers method in PaymentService

PaymentFactory This code shows the factory method that builds the strategy map and composes the decorator chain, centralising the creation of fully configured PaymentStrategy objects.

```
public PaymentFactory(OrderRepository orderRepository, @NotNull
 List<PaymentStrategy> strategies) {
  this.orderRepository = orderRepository;

  this.strategyMap = new HashMap<>();
  for (PaymentStrategy strategy : strategies) {
    PaymentType type = strategy.getType();

    // Ensure no duplicate strategies for the same payment type since we
 are using List injection by for loop
    if (this.strategyMap.containsKey(type)) { ... }

    this.strategyMap.put(type, strategy);
  }
}
// 2. Build decorated strategy chain based on requested type
public PaymentStrategy getStrategy(PaymentType paymentType) {

  PaymentStrategy rawStrategy = strategyMap.get(paymentType);

  if (rawStrategy == null) { ... }

  // 3. Wrap with decorators: Retry -> Validation -> Raw Strategy
  PaymentStrategy retryStrategy = new PaymentRetryDecorator(rawStrategy);
  PaymentStrategy validationStrategy =
  new PaymentValidationDecorator(retryStrategy, orderRepository);

  return validationStrategy;
```

```
27  }
```

Listing 30: PaymentFactory building strategy map and decorator chain

PaymentValidationDecorator This fragment highlights a concrete decorator that wraps a base PaymentStrategy to enforce business validation rules (such as order status and ownership) before the payment is executed.

```
1  public class PaymentValidationDecorator extends PaymentStrategyDecorator {
2
3    private final OrderRepository orderRepository;
4
5    // Constructor: Inject OrderRepository to access the database
6    public PaymentValidationDecorator(PaymentStrategy wrappedStrategy,
   OrderRepository orderRepository) {
7      super(wrappedStrategy);
8      this.orderRepository = orderRepository;
9    }
10
11   @Override
12   public PaymentResponse pay(@NotNull PaymentRequest request) {
13
14     log.info("Validating payment request for Order ID: {}",
   request.getOrderId());
15
16     if (request.getOrderId() == null) {
17       log.error("Validation failed: Order ID is missing.");
18       throw new IllegalArgumentException("Order ID cannot be null");
19     }
20
21     // Verify if the order exists in the database
22     Optional<Order> orderOpt =
   orderRepository.findById(request.getOrderId());
23     if (orderOpt.isEmpty()) { ... }
24
25     Order order = orderOpt.get();
26
27     // Prevent duplicate payments
28     // Only allow payment if the order status is 'PENDING'
29     if (order.getOrderStatus() != OrderStatus.PENDING) { ... }
30
31     log.info("Validation passed. Proceeding to payment logic.");
32
33     // Validation is successful, pass to the next layer (Retry Logic ->
   Real Payment)
34     return super.pay(request);
35   }
36 }
```

Listing 31: PaymentValidationDecorator enforcing business validation rules

9.3 Security and OWASP-inspired hardening

9.3.1 Description

Security is treated as a separate concern and implemented with Spring Security and JWT, following OWASP themes such as secure authentication, protection of sensitive data, and proper access control. SecurityConfig defines a stateless SecurityFilterChain that disables server-side sessions, registers the custom JwtAuthenticationFilter, and restricts access to management endpoints while keeping registration and login endpoints open. In AuthenticationService.register, the plaintext password from RegisterRequest is encoded with a BCryptPasswordEncoder before the Customer entity is saved, which reduces the risk if the database is leaked.

For API authentication, JwtUtil creates signed JWT tokens with a configurable secret and expiry time. JwtAuthenticationFilter reads the Authorization header, validates the token, loads the user through UserDetailsService, and places the authenticated user in the security context for later processing in controllers. TokenStorageService stores active tokens in Redis with a time-to-live value, supports token removal on logout, and is checked from the filter before a token is accepted. Together, these measures reduce the risk of broken authentication and session management and clearly separate anonymous guests from authenticated customers.

9.3.2 Implementation

SecurityConfig.securityFilterChain This snippet configures a stateless SecurityFilterChain with URL-based authorisation rules, registration and login whitelisting, OAuth2 login, and insertion of the custom JwtAuthenticationFilter.

```
1  public SecurityFilterChain securityFilterChain(@NotNull HttpSecurity http)
    throws Exception {
2
3     http.csrf(AbstractHttpConfigurer::disable)
4     .authorizeHttpRequests(auth -> auth
5     .requestMatchers(
6     "/api/auth/**",
7     "/hello",
8     "/error",
9     "/actuator/**",
10    "/oauth2/**",
11    "/login/oauth2/**",
12    "/api/rooms/search",
13    "/api/bookings/lock/**/status",
14    "/api/hotel/**"
15    ).permitAll()
16    .anyRequest().authenticated()
17    )
18    .sessionManagement(session ->
19    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
20    )
21    .oauth2Login(oauth2 -> oauth2
22    .userInfoEndpoint(userInfo -> userInfo
23    .userService(customOAuth2UserService)
24    )
```

```
    .successHandler(oAuth2AuthenticationSuccessHandler)
    )
    .authenticationProvider(authenticationProvider())
    .addFilterBefore(jwtAuthFilter,
   UsernamePasswordAuthenticationFilter.class);

    return http.build();
  }
```

Listing 32: SecurityConfig.securityFilterChain configuration

AuthenticationService.register and login methods This fragment shows secure registration and authentication, including BCrypt password hashing, delegation to AuthenticationManager, and JWT token generation for authenticated customers.

```
@Transactional
public LoginResponse register(@NotNull RegisterRequest request) {

  if (customerRepository.findByEmail(request.getEmail()).isPresent())
  throw new EmailAlreadyExistsException("Email already registered");

  var customer = Customer.builder()
  .name(request.getName())
  .email(request.getEmail())
  .passwordHash(passwordEncoder.encode(request.getPassword()))
  .phoneNumber(request.getPhoneNumber())
  .role(Role.USER)
  .membershipTier(MembershipTier.BRONZE)
  .enabled(true)
  .build();

  var savedCustomer = customerRepository.save(customer);
  var token = tokenGeneration(customer);

  return LoginResponse.builder()
  .token(token)
  .email(savedCustomer.getEmail())
  .name(savedCustomer.getName())
  .build();
}
```

Listing 33: AuthenticationService.register method

```
public LoginResponse login(@NotNull LoginRequest request) {

  try {
    var authentication = authenticationManager.authenticate(
    new UsernamePasswordAuthenticationToken(
    request.getEmail(), request.getPassword())
    );

```

```
    if (!authentication.isAuthenticated())
    throw new BadCredentialsException("Bad credentials");

    var customer = customerRepository.findByEmail(request.getEmail())
    .orElseThrow(() -> new UsernameNotFoundException("User not found"));

    var token = tokenGeneration(customer);

    return LoginResponse.builder()
    .token(token)
    .email(customer.getEmail())
    .name(customer.getName())
    .build();

  } catch (BadCredentialsException e) {
    throw new InvalidCredentialsException("Invalid email or password", e);
  }
}
```

Listing 34: AuthenticationService.login method

JwtAuthenticationFilter and TokenStorageService These snippets illustrate how each request is checked for a valid JWT, verified against Redis-backed token storage with TTL, and then translated into a Spring Security authentication in the SecurityContext.

```
protected void doFilterInternal(@NonNull HttpServletRequest request,
@NonNull HttpServletResponse response,
@NonNull FilterChain filterChain)
throws ServletException, IOException {

  final var authHeader = request.getHeader("Authorization");

  if (authHeader == null || !authHeader.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
  }

  try {
    final var jwt = authHeader.substring(7);

    if (!tokenStorageService.validateToken(jwt)) {
      logger.debug("Token not found in Redis or expired, denying access");
      filterChain.doFilter(request, response);
      return;
    }
  }
}
```

Listing 35: JwtAuthenticationFilter.doFilterInternal

```
/**
```

```java
 * Store a token in Redis with the specified expiration time
 *
 * @param email       The user's email (used as value)
 * @param token       The JWT token to store
 * @param expiration  Expiration time in milliseconds
 */
public void storeToken(String email, String token, Long expiration) {

  try {
    var key = TOKEN_PREFIX + token;
    redisTemplate.opsForValue().set(
    key, email, expiration, TimeUnit.MILLISECONDS);
    log.debug("Stored token for user: {} with expiration: {}ms",
    email, expiration);
  } catch (Exception e) {
    log.error("Failed to store token in Redis for user: {}", email, e);
    throw new RuntimeException("Failed to store token in Redis", e);
  }
}
/**
* Validate if a token exists in Redis (is not logged out or expired)
*
* @param token The JWT token to validate
* @return true if token exists in Redis, false otherwise
*/
public boolean validateToken(String token) {
  ...
}
```

Listing 36: TokenStorageService.storeToken

9.4 Database support and ORM

9.4.1 Description

Database support is built with Spring Data JPA, auditing, and Redis, and goes beyond a basic CRUD setup. All entities extend BaseEntityAudit, a @MappedSuperclass that defines an autogenerated identifier and the fields createdBy, updatedBy, createdAt, and updatedAt. JpaAuditingConfig enables Spring's auditing support and provides an AuditorAware implementation so these fields are filled automatically when entities are created or updated. This reduces duplicated code in entity classes and makes it easier to trace changes.

Domain entities use full JPA mappings instead of raw foreign key values. For example, Order has @ManyToOne associations to Customer and Room, implements PricingObserver, and defines several @Index annotations to optimise queries by customer, room, and status. Repositories such as RoomRepository, OrderRepository, and CustomerRepository extend JpaRepository and use derived query methods (such as findByRoomType_IdAndRoomStatus) instead of hand-written SQL. Persistence settings are externalised in application.properties and docker-compose.yml, which start a PostgreSQL container and connect the Spring Boot application through SPRING_DATASOURCE_ and SPRING_JPA_ environment variables. Redis is also configured for caching and token storage. This setup provides a portable, containerised data layer that supports

referential integrity, auditing, and efficient queries without exposing SQL directly in the application code.

9.4.2 Implementation

BaseEntityAudit This code shows the audited base class that provides createdBy, updatedBy, createdAt, and updatedAt fields using Spring Data JPA auditing annotations, so all subclasses automatically record who changed what and when.

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntityAudit extends BaseEntity {

  @CreatedBy
  @Column(name = "created_by", updatable = false)
  private String createdBy;

  @LastModifiedBy
  @Column(name = "updated_by")
  private String updatedBy;

  @CreatedDate
  @Column(name = "created_at", updatable = false)
  private LocalDateTime createdAt;

  @LastModifiedDate
  @Column(name = "updated_at")
  private LocalDateTime updatedAt;
}
```

Listing 37: BaseEntityAudit with JPA auditing support

```
@Entity
@Table(
name = "\"order\"", // Quoted because "order" is a SQL reserved keyword
indexes = {
  @Index(name = "idx_order_customer", columnList = "customer_id"),
  @Index(name = "idx_order_room", columnList = "room_id"),
  @Index(name = "idx_order_status", columnList = "order_status")
}
)
```

Listing 38: Order entity table definition and indexes

Order entity This snippet demonstrates a rich JPA mapping with @ManyToOne associations to Customer and Room, indexed columns for efficient queries, and an implementation of PricingObserver.update() that recalculates totalPrice when room prices change.

```
services:

# PostgreSQL Database
postgres: <7 keys>
```

```

# Redis Cache
redis: <7 keys>

# Spring Boot Application
app: <7 keys>

volumes: postgres-data, redis-data

networks:
hotel-network: <1 key>
```

Listing 39: docker-compose.yml services, volumes, and network configuration

docker-compose.yml (Postgres section and app environment) This fragment illustrates how PostgreSQL and the Spring Boot application are containerised and wired together via SPRING_DATASOURCE_ and SPRING_JPA_ environment variables, providing a portable ORM-backed data layer.

```
public class Order extends BaseEntityAudit implements PricingObserver {
  private Customer customer;
  private Room room;

  @Column(name = "check_in_date", nullable = false)
  private LocalDate checkInDate;

  @Column(name = "check_out_date", nullable = false)
  private LocalDate checkOutDate;

  @Column(name = "number_of_nights", nullable = false)
  private Long numberOfNights;

  @Column(name = "total_price", nullable = false, precision = 10, scale = 2)
  private BigDecimal totalPrice;

  private OrderStatus orderStatus = OrderStatus.PENDING;

  @Column(name = "check_in_code", unique = true, length = 8)
  private String checkInCode;

  @Column(name = "cancelled_at")
  private java.time.LocalDateTime cancelledAt;

  @Column(name = "cancellation_reason", length = 500)
  private String cancellationReason;

  /** Observer Pattern implementation ... */
  @Override
  public void update(float newPrice) { ... }
}
```

Listing 40: Order entity implementing PricingObserver

9.5 Monolith Architecture Pattern

9.5.1 Description

The system is implemented as a single deployable Spring Boot application (a monolith). This choice matches a proof-of-concept scope because it can reduce deployment and operational overhead compared with running multiple distributed services. All modules run in one process, so internal calls do not require network communication. This can support rapid development and simpler local debugging.

To keep the structure clear, we use a modular organisation inside the monolith. The code is separated into packages (controllers, service, repository, entity, security, config, dto). Dependencies are intended to follow a clear direction between these packages to limit coupling and keep responsibilities separated.

A limitation of this approach is that components cannot be scaled independently. The system can still scale by running multiple instances, but the full application is scaled as one unit. If the project scope grows, the current modular organisation can support future refactoring of major subsystems.

9.5.2 Implementation

The monolith is organised as layered packages with clear responsibilities:

- **controllers**: REST endpoints and request validation.
- **service**: business use cases and orchestration.
- **repository**: persistence access for PostgreSQL and Redis.
- **entity**: domain model and JPA mappings.
- **security/config/dto**: cross-cutting support and API boundary models.

9.6 Google Login

9.6.1 Description

Google Login provides an alternative sign-in option, where authentication is handled through an external identity provider instead of a local password-based flow. This can improve the sign-in experience for users and can reduce the amount of password handling required inside the application.

This feature is implemented using Spring Security OAuth2 login. After Google authentication succeeds, post-login processing is handled through a configured user service and a success handler. Access control remains role-based, so authorisation rules are applied in the same way regardless of the login method.

9.6.2 Implementation

The OAuth2 configuration is part of the security filter chain. The system configures a custom user service and a success handler for post-login actions.

```
1 http
2 .oauth2Login(oauth2 -> oauth2
3 .userInfoEndpoint(userInfo -> userInfo
4 .userService(customOAuth2UserService))
5 .successHandler(oAuth2AuthenticationSuccessHandler));
```

Listing 41: OAuth2 login configuration for Google sign-in

10 Recovered Architecture And Design Blueprints

10.1 Design-time Package Diagram

This figure shows the recovered design-time package diagram for the final implementation of the Hotel Reservation System. The concrete packages under com.example.hotelreservationsystem are grouped into four conceptual layers: view, controllers, Domain/Business Logic, and Infrastructure/Data, which follows the MVC architecture.

The view layer represents the external web and kiosk interfaces. These clients send HTTP/JSON requests to the backend and do not contain business logic. The controllers layer (package controllers) exposes REST endpoints, validates input, and delegates work to the service layer before returning HTTP responses.

The Domain/Business Logic layer holds the main application behaviour. The service and service.roomstate packages implement the key use cases and the reservation lifecycle. The entity package contains the JPA entities, while events and events.observers provide internal event handling. Cross-cutting mechanisms and business sub-modules (payment, pricing, notification) are grouped under pattern and base.

The Infrastructure/Data layer provides persistence and technical support. The repository package encapsulates database and Redis access, config wires these components into Spring Boot, security handles authentication and authorisation, and dto defines request and response objects at the API boundary.

Compared with the earlier analysis package diagram, this recovered structure is more detailed and matches the real code. The separation into view, controller, domain, and infrastructure layers is still in place, and dependencies flow downwards only—from view to controllers, from controllers to the domain layer, and from the domain layer to infrastructure.

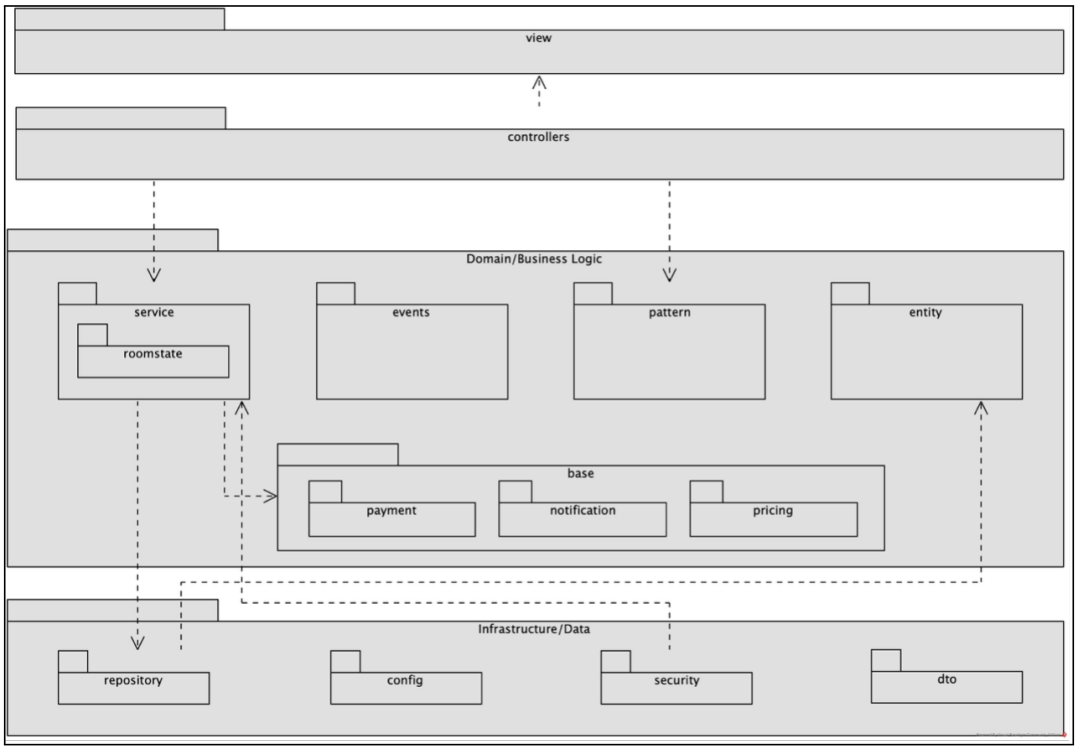


Figure 20: Design-time Package Diagram of the Hotel Reservation System

10.2 Design-time Class Diagram

10.2.1 Core entities

This design-time class diagram comes from the final JPA entity implementation and has been simplified and rearranged for readability. It shows a common audited base class, BaseEntityAudit, which is extended by Customer, Order, Hotel, Room, and RoomType so that identifiers and audit timestamps are handled in a consistent way.

The associations represent the main business relationships: a Customer can have many Order objects, each Order reserves one Room, a Hotel manages multiple Room objects, and every Room is linked to a specific RoomType. The diagram provides a compact view of the domain model implemented in the code and forms the basis for the higher-level services and patterns shown in the later diagrams.

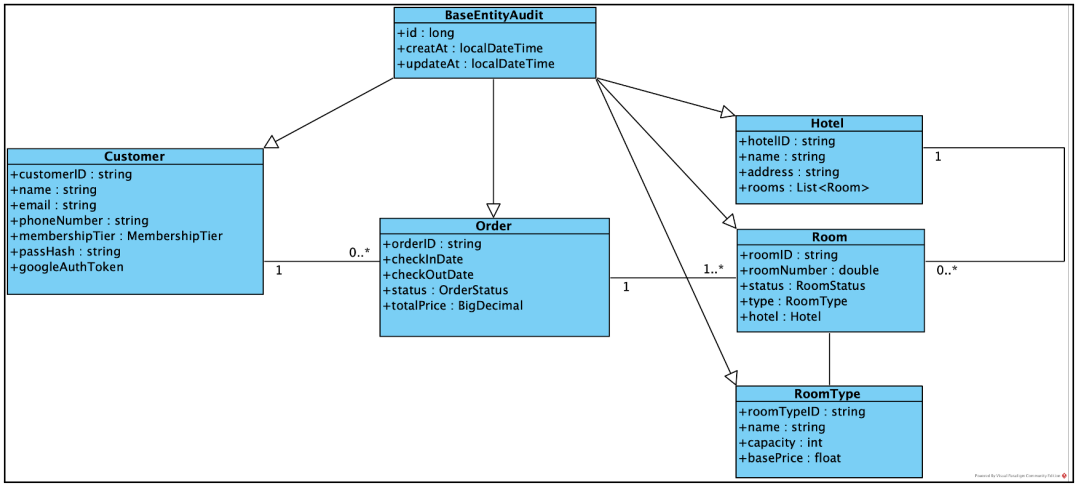


Figure 21: Design-time class diagram – Core entities

10.2.2 Core services

This design-time class diagram focuses on the core booking services. BookingService sits at the centre and coordinates the use cases for creating and cancelling bookings. It works with OrderRepository, RoomRepository, and CustomerRepository to store and load the required data. Infrastructure concerns are handled by dedicated services: BookingLockService (using Redis) prevents conflicting reservations, RoomSearchService and RoomService manage availability, and PaymentService processes payments. NotificationFactory is responsible for creating notification channels such as email or SMS.

This design keeps the main booking logic in a single application service that orchestrates repositories and supporting services, rather than spreading complex behaviour across controllers or entities.

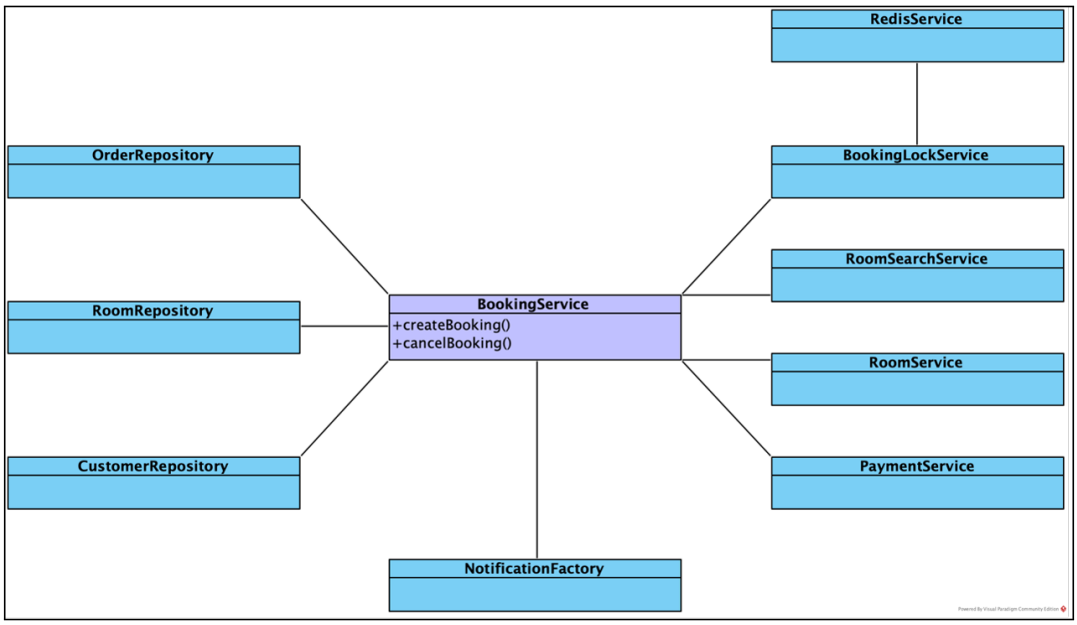


Figure 22: Design-time class diagram – Core services (Booking)

10.2.3 Payment patterns

This design-time class diagram focuses on the payment subsystem and shows how several design patterns are used in combination.

The Observer pattern groups PaymentObserver and its implementations (PaymentNotificationObserver, PaymentStatusUpdateObserver, PaymentAuditObserver) with PaymentService, which notifies observers when a payment completes so that related logic stays decoupled from the core flow.

The Factory pattern appears in PaymentFactory, which creates PaymentStrategy instances (such as CreditCardPaymentStrategy and PayPalPaymentStrategy) based on PaymentType and hides the details of how these objects are built. The Strategy pattern is expressed through the PaymentStrategy interface, which defines a common API for different payment workflows and allows the payment method to be chosen at runtime.

The Decorator pattern is used in PaymentStrategyDecorator and its concrete subclasses (PaymentValidationDecorator, PaymentRetryDecorator), which wrap a base strategy to add validation and retry behaviour without changing the original classes.

Together, these patterns make it possible to add new payment methods or cross-cutting concerns with minimal changes to PaymentService, and they support extensibility and maintainability in line with the open-closed principle.

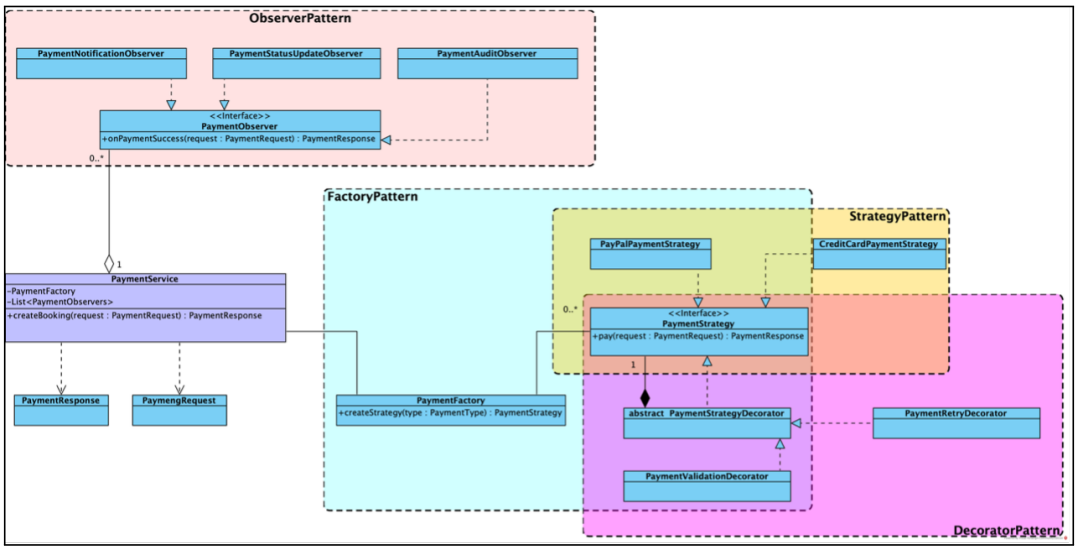


Figure 23: Design-time class diagram – Payment patterns

10.2.4 Pricing engine and order updates

This design-time class diagram shows the pricing subsystem and how it interacts with order management. The PricingComponent interface defines the calc() method, which RoomPricing implements to calculate the base room price. Extra behaviour is added through the abstract PricingDecorator and its concrete subclasses HolidayPricingDecorator, MembershipPricingDecorator, and OccupancyPricingDecorator. These decorators wrap other PricingComponent instances so that several pricing rules can be combined.

RoomPriceFactory builds an appropriate chain of decorators based on the RoomPriceTier, customer information, and room attributes, and returns a configured PricingComponent to the caller. For integration with orders, the PricingObserver interface defines a callback method. Order implements this interface and receives price updates via update(), which adjusts the totalPrice field when the calculated price changes.

This design combines the factory pattern for object creation, the decorator pattern for dynamic behaviour composition, and the observer pattern for communication between pricing and orders. In this way, pricing strategies remain flexible while the order implementation stays relatively simple.

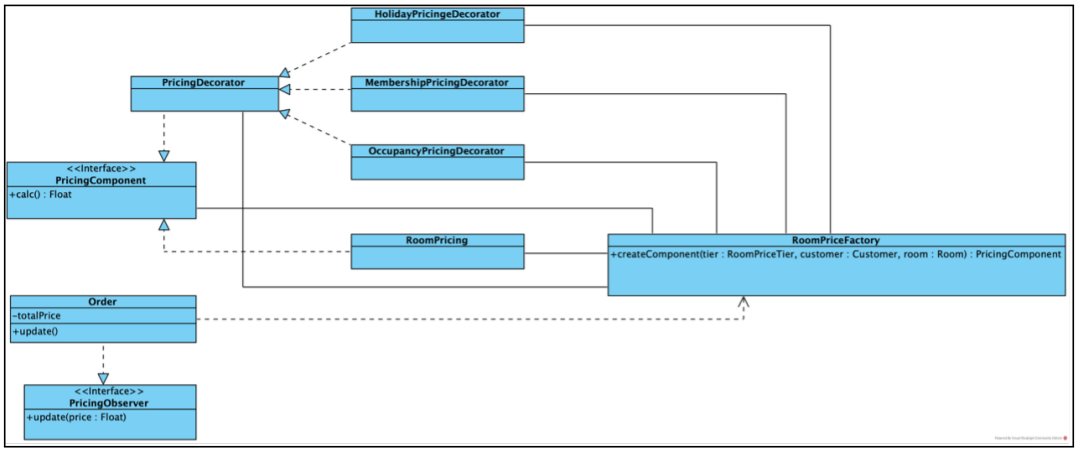


Figure 24: Design-time class diagram – Pricing engine and order updates

10.3 Sequence Diagram

10.3.1 Searching Room

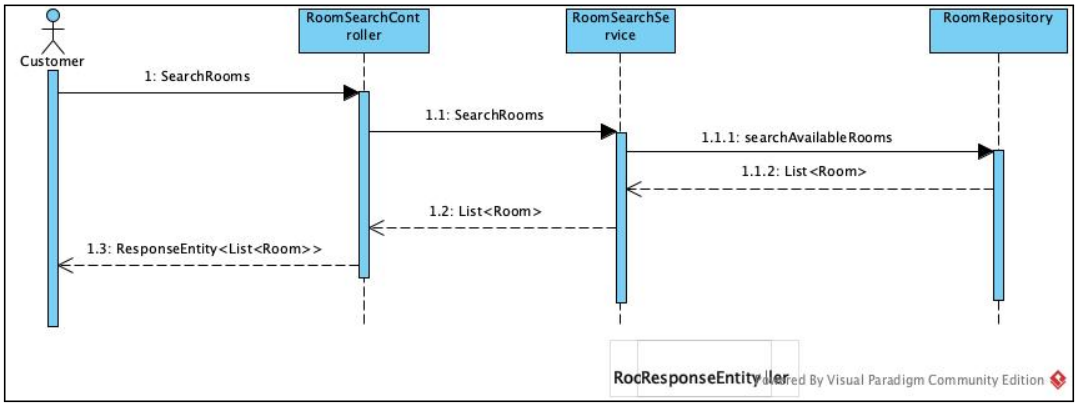


Figure 25: Searching Room sequence diagram

10.3.2 Booking Room

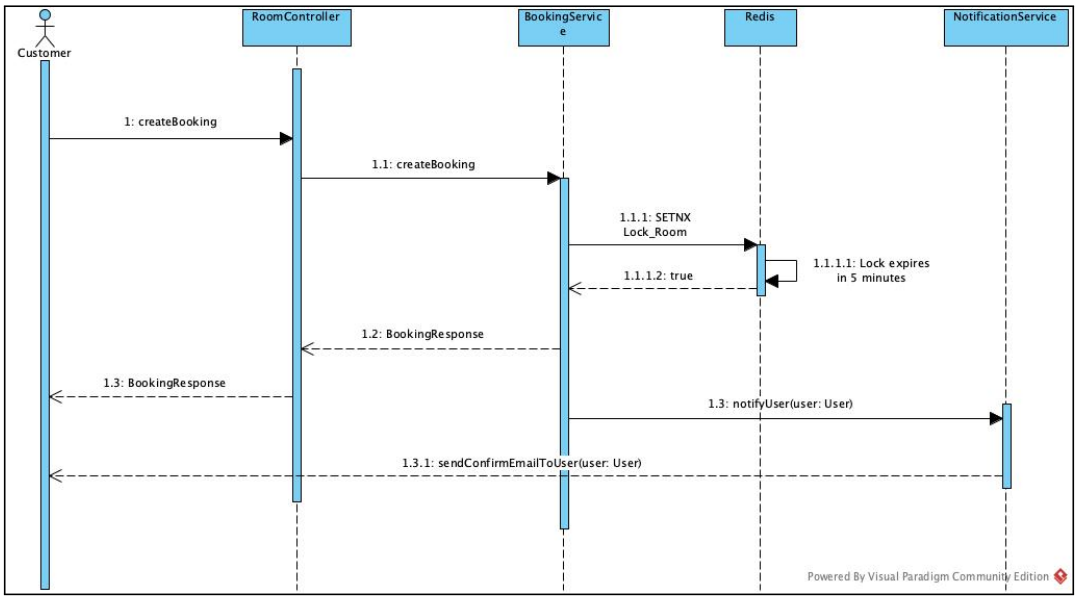


Figure 26: Booking Room sequence diagram

10.3.3 Payment Process

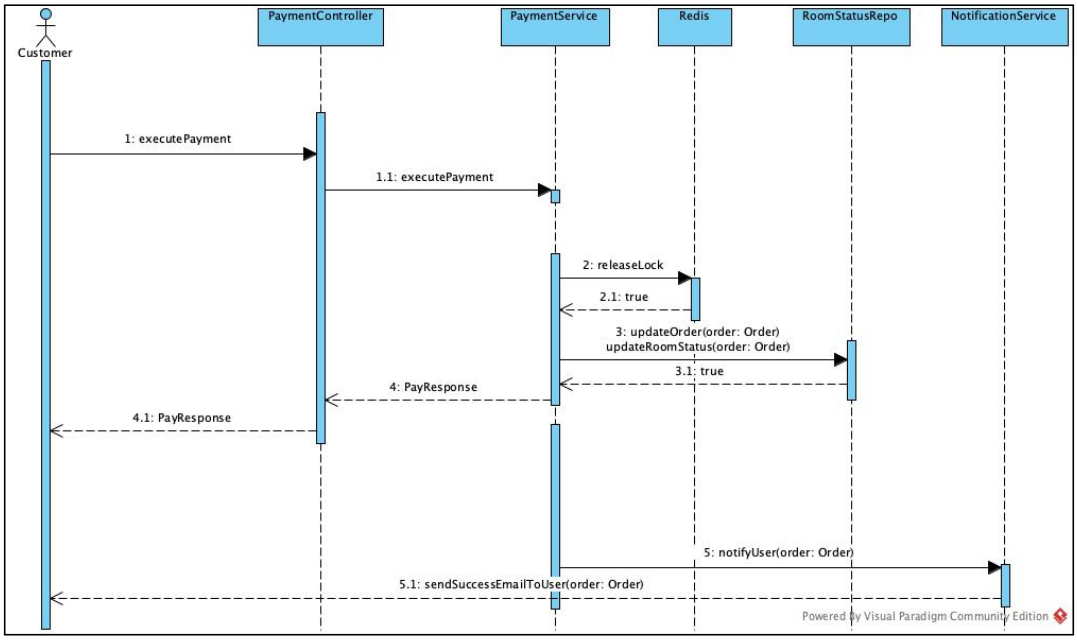


Figure 27: Payment Process sequence diagram

10.4 Deployment Diagram

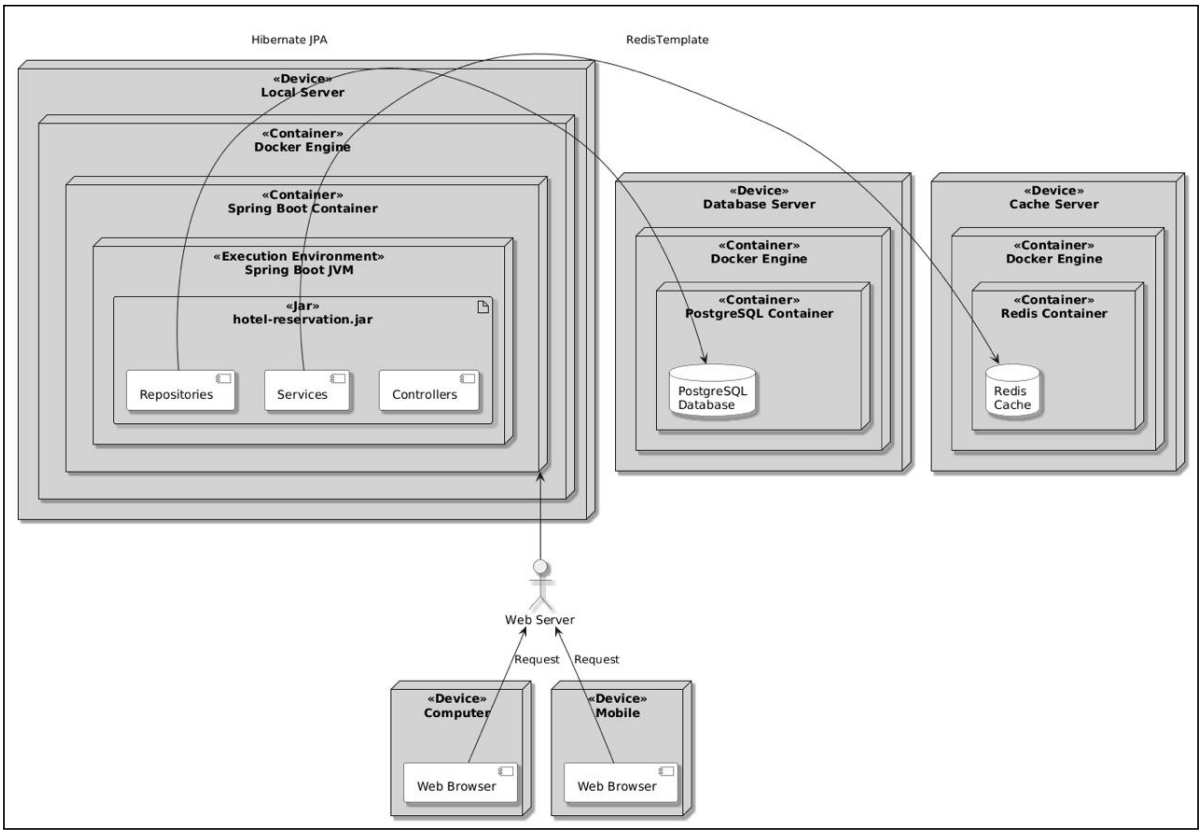


Figure 28: Deployment Diagram for the ABCD Hotels

10.5 Component Diagram

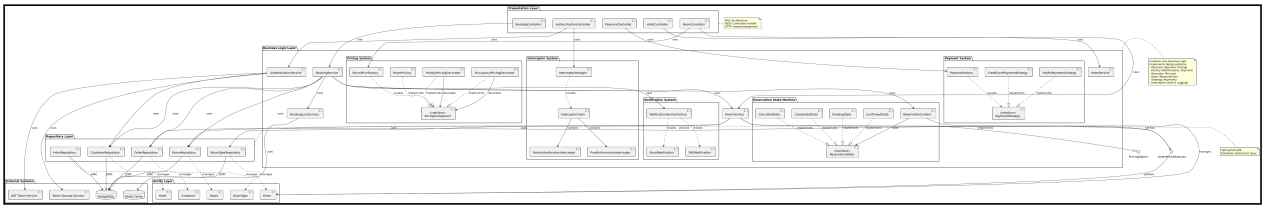


Figure 29: Component Diagram

11 Evaluation

Overall, the system design is suitable for a proof-of-concept. The application follows a Spring Boot monolith with Spring MVC and a layered package structure. This gives a clear separation between controllers, services, repositories, and entities. In general, this makes the code easier to read and change, because responsibilities are mostly grouped in the right place.

A positive point is the use of design patterns in areas where the design needs flexibility. For example, the payment module combines Strategy, Factory, Decorator, and Observer to separate payment methods, creation logic, validation/retry behaviour, and side effects such as notifications. This improves extensibility, because new payment methods or checks can be added with fewer changes to existing code. However, this also adds complexity. When patterns are combined, the control flow can become harder to trace, especially for new team members. In our work, some parts of the report describe the intent clearly, but some code snippets across sections can look inconsistent. This is a documentation weakness, because readers may not know which version matches the final code.

The diagrams provide strong support for communication. Use case diagrams make scope and user goals clear. Class and ER diagrams help explain data and relationships. Sequence diagrams clarify the order of calls for key flows such as searching, booking, and payment. Deployment and component diagrams help readers understand how the system runs. The UML is most effective when names, roles, and relationships match the implementation. A limitation in our documentation is that some diagrams are simplified for readability, but this is not always stated clearly. This can create confusion about what is exact versus what is an abstraction.

The architecture can be evaluated by checking quality attributes with evidence. Performance and concurrency can be assessed through load tests that simulate many users searching and booking at the same time, and by confirming that Redis locks prevent double booking under race conditions. Security can be assessed by verifying access rules, JWT validation, token expiry behaviour, and that logs do not expose sensitive data. Maintainability can be assessed by checking package dependency direction, size and complexity of core services, and whether changes can be made without touching many unrelated files.

To improve our work, the main need is stronger alignment between the report and the final implementation. We should make the documentation more consistent across sections, so each subsystem is described in the same style and level of detail. Code snippets should be checked against the final repository version, and diagrams should include a brief note when they are simplified. This will make the architecture easier to follow, easier to verify, and easier to maintain in later updates.

12 Reflection

12.1 Ismail Bahlaoui

At the beginning of this module, I was sceptical about my ability to contribute at the same level as my teammates because I do not have heavy industry experience. Most of my previous work was individual work and freelancing, so I had limited practice with large team-based projects. However, because my group members are senior and experienced, I received strong support and guidance throughout the semester.

Through this project, I learned how to work effectively in a team using version control, including feature branches, pull requests, and review-based merging. I also improved my understanding of how to transform design artefacts into implementation, especially how to connect UML diagrams to real Spring Boot code and package structure. In addition, my teammates helped me follow better software engineering practices, such as writing clearer code, keeping responsibilities separated, and applying design patterns in a structured way.

Overall, this project gave me a strong learning experience and increased my confidence in backend development and software design. Finally, I would like to thank my teammates for creating a supportive environment where everyone felt comfortable to ask questions, try new ideas, and improve their skills.

12.2 Dechao Zhan

I have worked as a frontend developer for several years, and this is my first time working as a backend developer. I started learning Java Spring Boot from scratch, and I learned a lot from this project. I set up the project structure and implemented basic dependencies such as PostgreSQL and Redis. I also learned how to use JPA to interact with the database. The Java Spring Boot framework is very powerful. It provides an easy way to build RESTful APIs, but there are many annotations. A system with many annotations can be confusing at times, and I tend to think this may harm the maintainability of the project in the long run. Another important thing I learned is how to implement design patterns in a backend project. This is quite different from my experience as a frontend developer. Previously, I used design patterns in a more informal way, without strictly following their definitions. Finally, I learned how to draw software diagrams such as sequence diagrams and component diagrams. I had never created these diagrams before, so this was a valuable learning experience for me.

12.3 Tan Le

At the beginning of the semester, I was unsure of how this module could be useful for me in an industrial context. However, after working on this team-based project, I believe I have become better as a software engineer, as I know how to apply design patterns in specific situations to optimise my project. Implementing interceptors from scratch taught me how to validate and control request flows before they reach business logic. Our biggest challenge was preventing double bookings, which we solved using Redis temporary locks during booking order and custom interceptors to validate requests before processing. Working in a team also taught me to spot logical gaps in requirements, especially around payment flows, which would affect staff procedures and risk management. This project connected academic concepts to industry practices, which gave me practical experience with Spring Boot REST APIs, ORM, and Redis session management that prepared me for further backend development-related roles.

12.4 Wenkai Zhu

At the beginning of this module, my background was mainly in IT consulting rather than software development,So I did not have the same level of industrial coding experience that some of my teammates had. Through the semester, I learned the basic ideas of software design, including how to think in terms of responsibilities, layers, and quality attributes. I also became more comfortable with drawing UML diagrams to communicate structure and behaviour, and with recognising and implementing core design patterns such as Strategy, Decorator, Factory Method, Observer, and State. Therefore, I tried to apply what I learned in my parts of our group project, and to reflect these ideas clearly in our report, class diagrams, and pattern descriptions. Overall, this project helped me bridge the gap between my previous consulting experience and more systematic backend design, and gave me more confidence on real-world Java/Spring systems.

13 Use of Generative AI

During this project, Generative AI tools were used as an assistive aid mainly for writing support and clarifying technical concepts. We did not keep a complete prompt log during development, so we report only the typical prompt categories used. All final design decisions, implementation work, experiments, and report claims were reviewed and validated by the team.

- **Grammar and clarity:** Rewrite a paragraph to improve grammar and readability while keeping the same meaning and technical terms.
- **LaTeX formatting:** Convert provided text into LaTeX using the existing report style (sections, tables, captions), without changing wording.
- **Design pattern explanation:** Explain a given design pattern (e.g., Strategy, Factory, Decorator, Observer, State, Interceptor) and relate it to a short code snippet or a package/module description.
- **Diagram support:** Suggest what to include in UML diagrams (use case, class, sequence, component, deployment) based on a described workflow, then we created the final diagrams ourselves using our chosen tools.
- **Testing and documentation:** Suggest unit test ideas (JUnit/Mockito) for a described service/controller, and propose how to document test intent and expected outputs.
- **Git and repository analysis:** Provide commands or scripts to compute metrics such as LOC per author, commits per author, or file ownership summaries.

References

[1] Bass, L., Clements, P. and Kazman, R. (2021) *Software Architecture in Practice*. 4th edn. Boston, MA: Addison-Wesley Professional.

[2] Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

[3] McGraw, G. (2006) *Software Security: Building Security In*. Upper Saddle River, NJ: Addison-Wesley.

[4] Walls, C. (2022) *Spring in Action*. 6th edn. Shelter Island, NY: Manning Publications.

[5] Perdita Stevens and Rob Pooley (1999). *Using UML: Software Engineering with Objects and Components*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA.

[6] Simon Bennett, Steve McRobb, and Ray Farmer (2005). *Object-Oriented Systems Analysis and Design Using UML*. McGraw-Hill Higher Education.

[7] Oracle (n.d.). *Java SE Documentation*. Available at: https://docs.oracle.com/en/java/javase/.

[8] VMware (Spring) (n.d.). *Spring Boot Reference Documentation*. Available at: https://docs.spring.io/spring-boot/docs/current/reference/html/.

[9] Refactoring.Guru (n.d.). *Design Patterns*. Available at: https://refactoring.guru/design-patterns.